

Competentie framework PL1 / CICS developer op z/OS level 1 definiëren en uitwerken met PoC binnen onderwijs-context.

Competentie framework PL1/CICS z/OS.

Robbe Van Herpe.

Scriptie voorgedragen tot het bekomen van de graad van
Professionele bachelor in de toegepaste informatica

Promotor: Dhr. L. Blondeel

Co-promotor: Dhr. M. Karademir

Academiejaar: 2025–2026

Eerste examenperiode

Departement IT en Digitale Innovatie .

**HO
GENT**

Woord vooraf

Mijn bachelorproef is gericht op nieuwe geïnteresseerde studenten voor mainframe. Ik hoop hen met deze Bachelorproef een duidelijke en bruikbare basis te kunnen geven, zoals ik die zelf ook graag gehad zou hebben toen ik eraan begon. Mijn doel is om de drempel lager te maken en de eerste stappen makkelijker en minder overweldigend te laten aanvoelen. Ik wil hiervoor zeker Leendert Blondeel bedanken. Dankzij zijn uitleg, feedback en de steun tijdens het jaar kon ik veel gericht werken. Ook de kansen die ik kreeg binnen de mainframe wereld dankzij Leendert Blondeel waren zeer waardevol en gaven me een kans om dieper in de onderwerpen zoals CICS en PL1 te gaan. Daarnaast wil ik Mehmet Karademir bedanken voor de technische hulp en begeleiding. Vooral wanneer ik ergens vastzat of met vragen zat, was zijn input en uitleg heel belangrijk om mijn visie op het juiste pad te houden. En ten slotte ook een grote dankjewel aan Euroclear om mij zo goed te ontvangen. Euroclear gaf me de kans om PL/1 en CICS in een echte omgeving te zien, waardoor ik er veel meer over kon leren. Dit maakte het geheel voor mij ook direct een stuk concreter. Door deze ervaring ben ik extra gemotiveerd om verder te groeien in dit veld.

Samenvatting

Mainframes zijn vandaag nog steeds een cruciaal onderdeel van de bedrijfskritieke infrastructuur binnen verschillende sectoren, variërend van financiële bedrijven zoals Euroclear tot de staalindustrie met ArcelorMittal. Toch kampt de mainframe industrie als geheel met een grote vergrijzings crisis. De laatste jaren neemt het aantal ontwikkelaars met kennis van PL/1 en CICS sterk af door vergrijzing en dalende instroom van nieuw talent uit het onderwijs.

Deze bachelorproef bekijkt hoe dit structureel probleem kan aangepakt worden voor PL/1 en CICS ontwikkelaars op z/OS. Het hoofddoel van deze bachelorproef is het ontwikkelen van een competentieframework, dat een overzicht geeft van kennis en vaardigheden die een startende PL/1 en CICS developer nodig heeft. Daarnaast ook het ontwikkelen van een proof of concept leerplatform dat studenten in staat moet stellen om deze competenties stapsgewijs op te bouwen.

Het framework werd opgesteld op basis van een literatuurstudie en semigestructureerde interviews met professionals uit Euroclear. De proof of concept is een online leerplatform dat is opgebouwd uit verschillende modules, gebaseerd op het opgestelde competentieframework. Binnen elke module verwerken studenten de leerstof aan de hand van theorie, praktische programmeeroefeningen en quizzen. Ten laatste werd deze leersite samen met het competentie framework gevalideerd door experts uit het werkveld en mede studenten van Hogent.

De resultaten van deze validatie tonen aan dat een gericht framework voor PL/1 en CICS ontwikkelaar een reële meerwaarde brengen voor zowel onderwijsinstellingen als bedrijven die nieuw talent willen aanwerven in hun legacy systemen. Door een praktijkgerichte aanpak te hanteren via een leerplatform met quizzen en oefeningen, wordt de leerstof niet alleen effectiever overgebracht, maar resulteert dit vooral in een veel betere kennisretentie bij de studenten. Het verkleint de skill gap en maakt het leren van deze complexe technologieën meer toegankelijk. Hierdoor krijgen startende ontwikkelaars een duidelijk groeipad om snel productief te worden in hun rol.

Inhoudsopgave

Lijst van figuren	viii
Lijst van tabellen	ix
Lijst van codefragmenten	x
1 Inleiding	1
1.1 Probleemstelling	1
1.2 Onderzoeksvraag	1
1.3 Onderzoeksdoelstelling	2
1.4 Opzet van deze bachelorproef	3
2 Stand van zaken	4
2.1 Het begrip mainframe	4
2.1.1 Definitie van een mainframe	4
2.2 Belangrijkste kenmerken van een mainframe	5
2.2.1 Veiligheid	5
2.2.2 Virtualisatie	5
2.2.3 Beschikbaarheid en betrouwbaarheid	6
2.2.4 Mainframe modernisatie	6
2.3 Mainframe-ontwikkeling in een hedendaagse context	7
2.4 Skills gap en vergrijzing als structureel probleem	7
2.5 Initiatieven om instroom van nieuw talent te vergroten	8
2.6 Onboarding van nieuwe ontwikkelaars in legacy omgevingen	8
2.7 Leerstrategieën en kennisoverdracht bij jonge IT professionals	9
2.7.1 Ervaringsgericht leren in een veilige context	9
2.7.2 Gamification	10
2.8 Competentieframeworks als fundament voor opleidingsopbouw	10
2.9 PL/I en CICS binnen z/OS-ontwikkeling	11
2.9.1 PL/I als ontwikkeltaal op het mainframe	11
2.9.2 CICS als transactieverwerker	11
2.9.3 PL/I en CICS als complementair geheel	12
2.10 Nood aan een PL/I en CICS-gericht framework	12
3 Methodologie	13
3.1 Opbouw van de methodologie	13
3.2 Fase 1: Analyse van competenties en requirements	13

3.3	Fase 2: Opstellen van het competentieframework	14
3.3.1	Opstellen initieel competentieframework	14
3.3.2	Interviews met professionals uit het werkveld	14
3.4	Fase 3: Ontwikkelen van de proof-of-concept	15
3.5	Fase 4: Valideren van het competentieframework en de PoC	15
4	Interviews	17
4.1	Structuur en opbouw van de interviews	17
4.2	Antwoorden van professionals	18
4.3	Conclusie van de interviews	19
5	Framework en PoC	20
5.1	Structuur en opbouw van het framework	20
5.2	Criteria voor het bepalen van de competenties	21
5.2.1	Basiskennis mainframe	21
5.2.2	Basis programmeerkennis	21
5.2.3	Praktijk gericht	21
5.2.4	De link met CICS	21
5.2.5	Hedendaagse IT landschap	22
5.3	Nodige Competenties	22
5.4	Volgende niveau Competenties	22
5.4.1	Geavanceerde PL1	23
5.4.2	CICS verdieping	23
5.4.3	Geavanceerde tooling en diagnostiek	23
5.4.4	Performantie optimalisatie en efficiëntie	23
5.4.5	Security architectuur	23
5.4.6	Modernisering	24
5.5	Van framework naar Proof of Concept	24
5.6	Structuur en opbouw van de PoC leersite	24
5.7	Bepalen van technologieën	24
5.7.1	Selectiecriteria	25
5.7.2	Gekozen basis van webtechnologieën	25
5.7.3	React als UI laag	25
5.7.4	Vite als build en ontwikkeltool	26
5.7.5	Tailwind CSS voor consistente en schaalbare styling	26
5.7.6	Content als JSON eenvoudig uitbreiden met nieuwe cursussen	26
5.7.7	Markdown en veilige HTML rendering	26
5.7.8	Lokale media zonder extra backend	27
5.7.9	Waarom geen backend of database?	27
5.7.10	Conclusie technologieën	27

5.8	Verschillende modules in de leersite	27
5.8.1	Origins and Evolution of PL/I and CICS	27
5.8.2	Environment Setup and ISPF Fundamentals	28
5.8.3	PL/I Core Concepts.	28
5.8.4	CICS Core Concepts	29
5.8.5	Debugging Basics	29
5.8.6	PL/I and CICS Integration Mastery	29
5.9	Bepalen opbouw van de modules	30
5.9.1	Theorie in modules	30
5.9.2	Oefeningen in modules	31
5.9.3	Quiz in modules	32
6	Conclusie	34
6.1	Antwoord op de onderzoeksvraag	34
6.2	Bijdrage aan het onderzoeksdomein	35
6.3	Kritische reflectie	35
6.3.1	Beperkingen van het onderzoek.	35
6.3.2	Verwachte resultaten van het onderzoek	36
6.4	Aanbevelingen voor verder onderzoek	36
A	Onderzoeksvoorstel	37
A.1	Inleiding	37
A.1.1	Probleemstelling en onderzoeksvragen	37
A.1.2	Doelgroep	38
A.1.3	Doelstelling	38
A.2	Literatuurstudie	38
A.3	Methodologie	39
A.3.1	Literatuurstudie en interviews	40
A.3.2	Opstellen framework en PoC	40
A.3.3	Rapporteren paper.	41
A.4	Verwacht resultaat, conclusie	41
	Bibliografie	43
	Competentie framework	46

Lijst van figuren

3.1	competentie framework.	14
5.1	Startscherm figuur.	28
5.2	Theorie figuur.	31
5.3	Oefeningen figuur.	32
5.4	Quiz figuur.	33

Lijst van tabellen

Lijst van codefragmenten

1

Inleiding

1.1. Probleemstelling

De IT-sector staat bekend als een snelle en dynamische omgeving waar men continu zoekt naar de nieuwste technologische oplossingen. Toch vormt een van de belangrijkste takken binnen deze industrie hierop een sterke uitzondering. Daar kiest men resoluut voor betrouwbaarheid en stabiliteit in plaats van de nieuwste technologieën. We spreken hier over de mainframe, deze machines worden al decennialang gebruikt om kritieke workloads te draaien voor bedrijven met extreem hoge transactievolumes. Dit zijn uiterst belangrijke organisaties binnen onze maatschappij zoals banken, verzekeraars en luchtvaartmaatschappijen. Om de grootste workloads te verwerken, maakt de mainframe nog altijd gebruik van oude technologieën zoals PL/I en CICS.

Het probleem waar ze de laatste 10 jaar tegenaan lopen, is de vergrijzing van het werkveld in deze sector. Universiteiten en hogescholen hebben hun curricula door de jaren heen steeds veranderd om de meest actuele onderwerpen aan studenten te kunnen bijbrengen. Bijgevolg komen technologieën als PL/I en CICS al ruim twintig jaar niet meer voor in het curriculum van moderne IT opleidingen. Door het tekort aan leeropportunities maar grote vraag voor nieuw talent bevinden studenten zich in een moeilijke positie door het tekort aan mogelijkheden tot zelfstudie. Deze bachelorproef is dan ook gericht op nieuwe studenten, voornamelijk die een stage zullen volgen binnen bedrijven zoals Euroclear of ArcelorMittal. Deze bedrijven maken gebruik van de PL/I en CICS technologieën.

Om deze studenten te helpen is een competentieframework nodig dat als basis kan dienen voor het ontwikkelen van leermateriaal.

1.2. Onderzoeksvraag

De bachelorproef bevat één hoofdvraag met drie deelvragen. De hoofdvraag luidt:

“Hoe kan een competentieframework voor PL/1 en CICS op z/OS ontwikkeld worden dat als basis kan dienen voor een leerplatform voor startende mainframeontwikkelaars bij Euroclear?”

Om deze hoofdvraag systematisch te kunnen beantwoorden, wordt deze opgesplitst in de volgende deelvragen:

- Welke competenties zijn essentieel voor een beginnende PL/1 en CICS developer binnen een z/OS omgeving? Hiermee wordt onderzocht wat de belangrijkste competenties zijn, zodat dit als basis gebruikt kan worden voor het opstellen van het framework.
- Hoe kan een competentieframework worden opgebouwd dat aansluit bij de actuele noden van het werkveld? Naast de essentiële competenties is het bekijken van de noden van het werkveld uiterst belangrijk om een compleet framework te kunnen opstellen dat actueel en meetbaar is.
- Hoe kan een proof of concept leerplatform op basis van dit competentieframework meetbaar bijdragen aan het verkleinen van de kenniskloof en het verkorten van de tijd tot inzetbaarheid van startende mainframeontwikkelaars? Hiermee wordt tot slot gekeken hoe een proof of concept leerplatform een concrete impact kan hebben in de actuele bedrijfswereld.

1.3. Onderzoeksdoelstelling

Het beoogde resultaat van de bachelorproef is het afleveren van een actueel en meetbaar competentieframework. Daarnaast wordt ook een proof of concept leerplatform ontwikkeld gebaseerd op dit competentieframework, waar studenten alle nodige vaardigheden kunnen leren om succesvol te zijn in het huidige werkveld rond PL/1 en CICS.

De succescriteria van deze bachelorproef zijn:

- Het competentieframework is actueel en relevant: de inhoud is afgestemd op realistische verwachtingen binnen organisaties zoals Euroclear die met PL/1 en CICS werken.
- Het competentieframework is meetbaar: elke competentie is beschreven met duidelijke niveaus of indicatoren, zodat het zichtbaar is wanneer een student de competentie beheerst.
- Het competentieframework is compleet en bevat alle nodige aspecten: het framework bevat ten minste alle aspecten dat een startende PL/1 en CICS developer nodig heeft binnen bedrijven zoals Euroclear.
- Het leerplatform implementeert het competentieframework: competenties zijn terug te vinden als leerdoelen of modules, en studenten kunnen gericht kiezen wat ze nog moeten leren.

- Het leerplatform bevat concrete leerinhoud en oefeningen: er moet een zeker aantal oefeningen zijn per hoofdstuk en een volledige leerinhoud zodat de student zich kan verdiepen en testen in de materie.
- Het leerplatform maakt vooruitgang zichtbaar: studenten kunnen op een eenvoudige manier zien welke onderdelen afgerond zijn en aan welke competenties nog gewerkt moet worden.
- Zowel het competentieframework als de proof of concept leersite zijn bruikbaar en begrijpelijk: een student kan ermee aan de slag zonder extra uitleg, en de link met stagevoorbereiding is duidelijk.

1.4. Opzet van deze bachelorproef

De rest van deze bachelorproef is als volgt opgebouwd:

In Hoofdstuk 2 wordt een overzicht gegeven van de stand van zaken binnen het onderzoeksdomein, op basis van een literatuurstudie.

In Hoofdstuk 3 wordt de methodologie toegelicht en worden de gebruikte onderzoekstechnieken besproken om een antwoord te kunnen formuleren op de onderzoeksvragen.

In Hoofdstuk 4 worden de interviews beschreven en worden de gebruikte interviewtechnieken besproken samen met de antwoorden van de professionals.

In Hoofdstuk 5 worden de technieken en technologieën voor het opstellen van de PoC besproken samen met een analyse van de inhoud.

In Hoofdstuk 6, wordt de conclusie gegeven en een antwoord geformuleerd op de onderzoeksvragen. Daarbij wordt ook een aanzet gegeven voor toekomstig onderzoek binnen dit domein.

2

Stand van zaken

Dit hoofdstuk bouwt verder op de probleemstelling uit de inleiding, en schetst de huidige stand van zaken rond de instroom van nieuwe mainframe PL/1 en CICS ontwikkelaars. Daarnaast wordt het belang van PL/1 met CICS en mainframes aangetoond op basis van actuele metingen in het werkveld. Tenslotte wordt er gekeken naar andere frameworks en waarom een competentieframework essentieel is voor het aanbrengen van PL/1 en CICS bij nieuw talent.

2.1. Het begrip mainframe

Een essentieel onderdeel om deze bachelorproef goed te kunnen volgen, is een duidelijk begrip van het concept "mainframe". De term wordt in de praktijk soms gebruikt alsof het gewoon om een zeer grote server gaat, maar dat beeld is eigenlijk te beperkt. Binnen de literatuur en documentatie over enterprise computing verwijst een mainframe naar een specifiek type platform dat ontworpen is voor continuïteit, grootschalige verwerking, en hoge betrouwbaarheid Spruth, [2010](#).

2.1.1. Definitie van een mainframe

Een mainframe kan omschreven worden als een enorm sterke computer, maar in tegenstelling tot andere servers is zijn doel heel specifiek. Het doel is het verwerken van enorm veel transacties op een zeer betrouwbare manier. Een mainframe is gebouwd om de downtime zo veel mogelijk te verkleinen om bedrijfskritieke toepassingen steeds draaiende te houden.

2.2. Belangrijkste kenmerken van een mainframe

2.2.1. Veiligheid

Een van de belangrijkste kenmerken van een mainframe is zijn hoge securability, wat besproken wordt in verschillende studies en papers van onder andere Spruth, 2010 en Ebbers e.a., 2005. Hoewel een mainframe is ontworpen met de architectuur om maximale veiligheid te faciliteren, mag dit niet verward worden met secure by design. Het systeem is uiteindelijk slechts zo veilig als de specifieke beveiligingsmaatregelen die erop zijn geconfigureerd Buecker e.a., 2016. Maar over het algemeen is de mainframe in veel bedrijven het meest veilige systeem tegen cybercriminelen, mede doordat IBM de hardware en software ontwikkelt, wat mogelijk maakt om beveiligingen op alle niveaus te plaatsen Buecker e.a., 2016.

Een voorbeeld van het hoge niveau aan veiligheid is dat vanaf de Z16 mainframe de beveiliging kwantum veilig is. De Z16 was het eerste systeem in de industrie die kwantumcryptografie over verschillende lagen van firmware toepaste. Dit werd bereikt door het toepassen van algoritmes bij versleuteling die geselecteerd zijn door NIST als post kwantum standaard Bradbury en Hess, 2021. Zo is de mainframe met zijn beveiligingsalgoritmes bestand tegen toekomstige kwantumcomputers, wat een extra troef is tegenover andere systemen.

2.2.2. Virtualisatie

Een tweede kenmerk van een mainframe is de ondersteuning voor virtualisatie door het gebruik van logische partities (LPARS). Deze LPARS zijn logische partities die een fysieke machine opsplitsen in meerdere compleet onafhankelijke virtuele systemen Buecker e.a., 2016. De mainframe wordt dan op softwareniveau verdeeld zodat elke LPAR over zijn eigen resources beschikt. Daarnaast kan iedere LPAR zijn eigen besturingssysteem draaien wat zorgt voor zeer hoge flexibiliteit.

Deze virtualisatielaag brengt dan enkele concrete voordelen. Ten eerste laat het toe om workloads heel flexibel te verdelen tussen verschillende partities, wat zorgt voor optimaal gebruik van de resources. Een tweede voordeel dat een virtualisatielaag biedt is dat fouten binnen een LPAR beperkt blijven tot zijn resources. Naast deze voordelen voor het systeem en de workloads geeft het ook de mogelijkheid om de machine op te delen in verschillende omgevingen. Deze omgevingen kunnen dan op één machine dienen voor ontwikkeling of testomgeving, wat organisaties extra flexibiliteit geeft bij het beslissen van hoeveel machines ze nodig hebben. Door het opdelen van het systeem met een tussenlaag kan er ook extra beveiliging aan gebracht worden tussen deze lagen. Hierdoor is het gebruik van logische partities binnen een mainframe heel voordelig binnen grote organisaties.

2.2.3. Beschikbaarheid en betrouwbaarheid

Een derde en zeer belangrijke eigenschap van de mainframe is de zeer hoge betrouwbaarheid en beschikbaarheid J. Raju, [2024](#). Doordat IBM de hardware en de software ontwikkelt kan men veel dieper gaan en veel lagen van veiligheid inbouwen. Volgens IBM is de uptime dan ook 99,999 procent wat uitkomt op een 5 minuten per jaar van downtime. Dit is veel lager dan gedistribueerde systemen en is vaak nog een onderschatting met nieuwe systemen zoals de Z17(nieuwste mainframe) die wel 99,9999999 procent uptime kunnen hebben. Hierdoor is dit een van de meest geschikte machines voor bedrijfskritieke processen op te draaien binnen grote organisatie.

Om deze hoge uptime te kunnen garanderen maakt IBM gebruik van software en hardware features die samen in harmonie werken om tot een heel betrouwbaar systeem te leiden Spruth, [2010](#). Enkele voorbeelden van deze features zijn: Op het laagste niveau de Recovery Unit(RU), dit is een hardwarecomponent die geïntegreerd is in de chip zelf. Wanneer zich een fout voordoet in het systeem, bijvoorbeeld een bitflip of een defecte transistor detecteert en isoleert de RU dit probleem onmiddellijk Lascu e.a., [2022](#). De getroffen component wordt automatisch vervangen door een backup component op dezelfde chip. Hierdoor is de impact op het complete systeem minimaal en kunnen andere processen ongestoord verder draaien.

Een niveau hoger spreken we van de parallel sysplex architectuur. Hierbij werken verschillende mainframes samen als één logisch systeem en delen ze databases en resources Ebbers e.a., [2012](#). Door deze verbondenheid tussen machines kan in het geval dat er één uitvalt de workload verplaatst worden naar een ander systeem. Dit herstel op een ander systeem verloopt heel snel en zonder dataverlies. De hersteltijd kan enkele seconden tot minuten duren.

Op het hoogste niveau staat IBM GDPS(Geographically Dispersed parallel sysplex). Aan de hand van deze technologie is een failover tussen verschillende gescheiden datacenters mogelijk N. Raju e.a., [2024](#). Dit systeem is in staat om de replicatie van data en overschakeling van werklasten tussen verschillende locaties te controleren. In geval van een cyberaanval of stroompanne op een bepaalde site is dit een laatste maar drastische oplossing om het systeem draaiende te houden.

Samen vormen deze drie niveaus voor een zeer robuuste architectuur waarbij gelocaliseerde fouten nooit kunnen uitgroeien tot problemen die het systeem stoppen. Grote financiële instellingen rekenen nu juist op deze betrouwbaarheid om er voor te zorgen dat de financiële wereld niet in elkaar valt.

2.2.4. Mainframe modernisatie

Een vierde en steeds meer belangrijke eigenschap van de mainframe is de integratie met artificiële intelligentie. Huidige systemen hebben deze integratie rechtstreeks ingebouwd op de processor zelf. Deze technologie is mogelijk gemaakt door de IBM telum chip zoals beschreven in de paper over de ai accelerator op de

IBM telum processor van Lichtenau e.a., [2022](#).

De Telum chip is de eerste processor met een geïntegreerde AI accelerator voor krachtige servercomputers. Het grote voordeel van een geïntegreerde AI accelerator op de chip is de mogelijkheid om data te verwerken zonder dat deze de chip moet verlaten. Dit laat toe dat AI modellen zoals transactiefraudedetectie kunnen draaien zonder dat hierdoor de transactie lang moet wachten.

Hierdoor is het mogelijk om fraudedetectie te draaien voor iedere transactie, in plaats van het gebruik van steekproeven. Grote financiële instellingen kunnen hierdoor elke creditcard transactie laten controleren op fraude zonder dat dit zorgt voor merkbare vertraging.

In de nieuwste IBM systemen is deze technologie nog verder ontwikkeld met de tweede generatie Telum chips en de Spyre accelerator. Door deze evolutie is de rekenkracht tot wel vier maal vergroot, wat de mainframe in staat stelt om krachtigere algoritmes te draaien op iedere transactie.

2.3. Mainframe-ontwikkeling in een hedendaagse context

Hoewel mainframe-technologie vaak als verouderde technologie wordt beschouwd, blijft door voorgaande kenmerken de mainframe een cruciaal onderdeel om bedrijfskritische processen te draaien in bepaalde sectoren. Organisaties die sterk inzetten op betrouwbaarheid, beschikbaarheid en voorspelbare verwerking blijven daarom investeren in z/OS-omgevingen. Voor deze organisaties wordt de mainframe gezien als een gespecialiseerd platform voor hun bedrijfskritieke toepassingen. Waar moderne systemen vaak gericht zijn op flexibiliteit en snelle iteratie, ligt de sterkte van de mainframe juist in zijn stabiliteit en continue beschikbaarheid. Door dit blijvend belang van mainframes binnen deze organisaties is de nood aan ontwikkelaars om deze te onderhouden nog steeds even groot.

Maar wat wordt nu juist bedoeld met een ontwikkelaar in deze mainframecontext. Een mainframe developer is een softwareontwikkelaar die applicaties op de mainframe analyseert, onderhoudt, uitbreidt en test. Voor PL/I en CICS ontwikkelaars betekent dit concreet dat zij businesslogica analyseren en aanpassen, transactiestructuren begrijpen en onderzoeken van fouten en het oplossen hiervan binnen legacy code omgevingen. In deze rol wordt er dan niet enkel programmeerkennis verwacht maar ook een grote portie aan inzicht in de architectuur, processen en historische opbouw van het systeem. Voor het onderhouden van deze systemen heeft men dus profielen nodig met enige ervaring in het onderhouden van legacy systemen, en net deze worden steeds moeilijker te vinden.

2.4. Skills gap en vergrijzing als structureel probleem

Een terugkerend thema in de literatuur is dat de mainframe industrie geconfronteerd wordt met vergrijzing en hierdoor een verlies aan kennis tegemoet gaat. Sharma

toont aan dat het aanleren van mainframevaardigheden in het onderwijs steeds moeilijker wordt om te verantwoorden en hierdoor het aantal instapklare profielen aan het dalen is in de mainframe industrie (Sharma & Murphy, 2011). Ook Ngo-Ye beschrijft hoe de kloof tussen academische curricula en de verwachtingen van bedrijven blijft groeien (Ngo-Ye & Choi, 2018).. Dit zorgt dan voor een probleem bij het vinden van geschikte starters om deze legacy systemen te onderhouden.

Het probleem wordt vergroot doordat slechts een beperkt aantal onderwijsinstellingen mainframegerichte opleidingsonderdelen aanbieden. Hierdoor is de instroom van nieuw talent beduidend laag, wat zorgt voor een groot probleem want de vraag in sectoren zoals de financiële wereld ligt hoger dan ooit.

Niet alleen is er een tekort aan mainframegerichte opleidingen maar ook een tekort aan motivatie bij studenten. Zij gaan sneller kiezen voor moderne technologieën die als toekomstgericht worden gezien. Door dit kleiner aanbod aan studenten en verandering van focus bij onderwijsinstellingen is de instroom van nieuw talent voor de mainframe steeds aan het verkleinen (Phillips e.a., 2013). Hierdoor ontstaat een vicieuze cirkel, minder onderwijsaanbod leidt tot minder interesse wat de instroom nog verder doet dalen. Voor organisaties zoals Euroclear betekent dit een reëel risico op kennisverlies wanneer ervaren medewerkers op pensioen gaan en hun expertise onvoldoende kan worden overgedragen naar de nieuwe werknemers.

2.5. Initiatieven om instroom van nieuw talent te vergroten

Voor dit probleem op te lossen en de kloof te verkleinen zijn er verschillende initiatieven ingezet de laatste jaren om de toegankelijkheid van leermateriaal te vergroten. Programma's zoals IBM Z Xplore verlagen de drempel door deelnemers stapsgewijs kennis te laten opbouwen en die vooruitgang zichtbaar te maken via badges of certificaten. Door het gebruik van deze certificaten kunnen ze een meetbaar bewijsstuk aanbrengen van hun vaardigheden en daarnaast werkt dit heel motiverend. Ook leertrajecten die dichter aansluiten bij het werkveld zoals IBM apprenticeships. Hier wordt gebruik gemaakt van rolgerichte verwachtingen en praktische begeleiding, waardoor een duidelijk pad gecreëerd wordt als voorbereiding op een bepaalde job (IBM, 2023c). Deze initiatieven tonen dat er een duidelijke nood is aan gestructureerde leerpaden, maar ze benadrukken ook dat er een nood is aan validatie en een overzicht van de nodige competenties.

2.6. Onboarding van nieuwe ontwikkelaars in legacy omgevingen

Met dit structureel tekort aan nieuw talent is het heel belangrijk om de startende ontwikkelaars zo effectief mogelijk te laten integreren in een team. Dit begint al

bij de onboarding waar enkele punten cruciaal zijn voor het opbouwen van kennis en vertrouwen bij een nieuw teamlid. Literatuur over developer onboarding (Britto, 2017) toont aan dat de instroom van nieuwe ontwikkelaars niet enkel afhangt van technische opleidingen, maar dat er meer aspecten dit beïnvloeden. Het gestructureerd inwerkingsproces blijkt een soms even groot belang te hebben als de technische skills. Britto (Britto, 2017) toont aan dat onboarding van developers in legacy softwareteams opgebouwd is uit drie delen: leren, zelfvertrouwen en sociale integratie binnen het team. Uit deze studie blijkt dat de eerste taken die een ontwikkelaar krijgt sterk kan beïnvloeden hoe snel een nieuwe ontwikkelaar productief kan worden. Daarnaast is het belang van goede initiële begeleiding niet te onderschatten. Een zelfverzekerde en goed begeleide beginner gaat veel sneller complexe code architectuur snappen doordat ze fouten durven te maken. Daarnaast is het communiceren met ervaren medewerkers een van de punten die een nieuwe ontwikkelaar, het meeste kan helpen om de complexe systemen van hun bedrijf te snappen. Dit is zeer relevant bij complexe legacy systemen, waarbij historische kennis noodzakelijk is om de huidige werking te begrijpen. Om deze technische basis en historische kennis vlot over te brengen, volstaat de klassieke schoolse aanpak echter niet. Er is nood aan gerichte leerstrategieën die specifiek aansluiten bij de behoeften van jonge professionals.

2.7. Leerstrategieën en kennisoverdracht bij jonge IT professionals

Om de instroom en onboarding van nieuw talent in een mainframeomgeving succesvol te laten verlopen, volstaat het niet om enkel technische documentatie aan te bieden. De manier waarop jonge professionals leren, vereist een specifieke aanpak. Uit recent onderzoek blijkt dat software engineering educatie het meest effectief is wanneer theorie direct gekoppeld wordt aan de praktijk via experiential learning (Holmes e.a., 2018).

2.7.1. Ervaringsgericht leren in een veilige context

Experientiële leertrajecten bieden startende ontwikkelaars de kans om aan de slag te gaan met authentieke taken en reële projecten uit de industrie (Holmes e.a., 2018). Binnen een mainframecontext betekent dit dat starters de theorie (zoals z/OS commando's of CICS transacties) direct moeten kunnen toepassen in een veilige testomgeving. Dit actieve proces van interactie en reflectie versnelt niet enkel de leercurve, maar helpt ingenieurs ook om complexe ontwerpprincipes en beperkingen in de praktijk sneller te begrijpen (Sotaquirá-Gutiérrez e.a., 2024).

2.7.2. Gamification

Tot slot leren moderne IT professionals het vlotst wanneer de theorie niet overwelddigt, maar juist motiveert via snelle feedback. Een goed voorbeeld hiervan is microlearning, waarbij de leerstof wordt opgesplitst in kleine behapbare blokken. Dit blijkt in de praktijk heel effectief om technische vaardigheden aan te leren en gerichte kennisgaten snel op te vullen (Granatella, [z.d.](#)). Als je deze aanpak ook nog combineert met gamification, zoals het behalen van badges in IBM Z Xplore (zie sectie [2.5](#)), raken starters sneller betrokken. Hierdoor blijft de motivatie om zichzelf te blijven ontwikkelen ook op de lange termijn behouden (Di Nardo e.a., [2024](#)).

Hoewel deze didactische methoden de *manier waarop* starters leren optimaliseren, vereisen ze wel een duidelijke inhoudelijke structuur. Het moet helder zijn welke specifieke vaardigheden en domeinkennis in deze leermodules of samenwerkingssessies aan bod moeten komen. Hier biedt een competentieframework de benodigde houvast.

2.8. Competentieframeworks als fundament voor opleidingsopbouw

Een competentieframework vertrekt vanuit een duidelijke definitie rond het begrip competentie. In de literatuur wordt een competentie niet beperkt tot het bezitten van kennis maar als een combinatie van kennis, vaardigheden en attitudes die iemand in staat stelt om effectief te kunnen functioneren in een professionele rol.

Een competentieframework maakt aan de hand van een overzicht duidelijk welke kennis en vaardigheden verwacht worden voor een bepaalde rol en helpen aan de hand van leerdoelen om de student een duidelijk stappenplan te bezorgen.

Voor mainframe profielen bestaan er al meerdere competentieframeworks. IBM heeft bijvoorbeeld enkele frameworks zoals IBM Z Systems Administrator Level 1 en Level 2 gepubliceerd (IBM, [2023a](#), [2023b](#)). Deze bevatten bruikbare elementen zoals opbouw en formulering van competenties, maar zijn hoofdzakelijk gericht op de systeem en beheerkant van de mainframe. Voor organisaties die vooral applicatieontwikkeling op z/OS willen ondersteunen is de meerwaarde daarom beperkt.

Daarnaast bestaat er een breder development georiënteerd framework van IBM om Z/OS ontwikkelaars algemene skills aan te brengen. Het Application Developer on IBM Z competentieframework (IBM Apprenticeship Program, [2023](#)) geeft een overzicht van belangrijke onderwerpen voor z/OS development, maar blijft op verschillende punten te algemeen om gebruikt te worden als framework voor PL/I en CICS developers binnen een bedrijf als Euroclear. De focus van dit framework ligt te veel op COBOL development, maar de structuur en aanpak om development te leren zijn wel twee aspecten die een grote meerwaarde zijn voor het ontwerpen van een nieuw PL/I en CICS development competentieframework.

2.9. PL/I en CICS binnen z/OS-ontwikkeling

De technologieën die worden gebruikt op de mainframe zijn historisch verbonden aan de architectuur van de hardware zelf. Twee technologieën die heel belangrijk zijn binnen mainframe development zijn PL/I en CICS. Samen vormen ze de ruggegraat binnen veel organisaties voor hun bedrijfskritieke applicaties op IBM Z systemen.

2.9.1. PL/I als ontwikkeltaal op het mainframe

Naast Cobol en Java is PL/I (Programming Language One) een van de meest gebruikte talen voor ontwikkeling op het mainframe. PL/I werd in 1964 door IBM ontwikkeld met als doel een programmeertaal te maken die wetenschappelijke toepassingen kon maken maar ook zakelijke.

Je ziet PL/I bijvoorbeeld in de financiële sector bij organisaties zoals Euroclear, maar ook in andere omgevingen zoals in de staalindustrie bij ArcelorMittal. Dat is niet toevallig, de taal is gebouwd om grote hoeveelheden transacties efficiënt te verwerken net zoals Cobol Dau e.a., [2024](#). IBM probeerde de wiskundige kracht van Fortran en de zakelijke kracht van Cobol te combineren tot één taal die alles kan. Hierdoor is PL/I iets minder business georiënteerd, waardoor sommige developers het iets toegankelijker vinden om mee te starten.

Op zichzelf zijn PL/I en Cobol relatief toegankelijke talen, maar in de praktijk kom je ze vooral tegen in legacy omgevingen. Daar bevindt zich nu net de complexiteit en uitdaging voor nieuwe developers. De legacy systemen zijn door de jaren heen uitgebreid en aangepast waardoor de totale complexiteit sterk is toegenomen.

Dat heeft ook te maken met de rol die deze applicaties spelen binnen financiële instellingen, daar draaien ze vaak de meest kritieke workloads. Voor het veranderen van deze kritieke toepassingen in productie zijn de regels vaak heel beperkend. Door deze heel strikte en beperkte manier van werken worden nieuwe problemen soms opgelost met workarounds of minimale aanpassingen. Hierdoor zijn oplossingen niet altijd optimaal en de technische schuld blijft verder groeien.

2.9.2. CICS als transactieverwerker

CICS (Customer Information Control System) is een middlewareplatform dat werkt als een OLTP (online transactie verwerkings systeem) tussen het z/OS besturings-systeem en de bedrijfsapplicaties Horswill en Members of the CICS Development Team at IBM Hursley, [2000](#). Het werd in 1968 ontwikkeld door IBM als een tijdelijke oplossing voor System/360 machines. Desondanks is deze technologie uitgegroeid tot een van de grootste softwareplatformen binnen de mainframe industrie. Vandaag verwerkt deze technologie meer dan tientallen miljarden transacties per dag, en wordt gebruikt in verschillende industrieën. CICS wordt ingezet in de financiële wereld bij banken en verzekeraars maar ook in de gezondheidszorg, overheid en winkelketens.

Het belangrijkste kenmerk dat CICS relevant heeft gehouden is de hoge flexibiliteit en taalondersteuning. CICS is compatibel met verschillende talen zoals COBOL, PL/I, Java en C++, wat het makkelijk laat integreren met nieuwe en bestaande applicatielagen.

2.9.3. PL/I en CICS als complementair geheel

PL/I en CICS vormen samen een complementaire basis binnen mainframearchitectuur. PL/I implementeert de businesslogica en CICS de transactielaag die de businesslogica naar de eindgebruiker brengt. Zonder deze samenhang van de technologieën zouden veel kritieke applicaties in de financiële sector niet kunnen functioneren op de schaal en betrouwbaarheid die vandaag vereist is.

De samenwerking van deze technologieën loopt in lijn met de hoofddoelen van een mainframesysteem. Het zorgt voor betrouwbaarheid en schaalbaarheid met een sterke focus op het onderhouden van oude en nieuwe toepassingen.

2.10. Nood aan een PL/I en CICS-gericht framework

De bestaande frameworks tonen aan dat het opbouwen van een competentieframework rond IBM Z mogelijk is, maar het zijn steeds rolspecifieke frameworks. Voorgaande frameworks zijn niet gericht op PL/I en CICS developers, met als voorbeeld het sysadmin framework van IBM (IBM, [2023a](#), [2023b](#); IBM Apprenticeship Program, [2023](#)).

Het bewezen belang van deze frameworks zijn een duidelijk beeld geven van de competenties die een student moet beheersen om succesvol te zijn in het huidige werkveld. Hierdoor gaat de student sneller inzetbaar worden binnen teams en is de skill gap waar interne training voor nodig is zo klein mogelijk (Ngo-Ye & Choi, [2018](#); Sharma & Murphy, [2011](#)). Voor bedrijven zoals Euroclear waar kennisborging en betrouwbaarheid centraal staan is een framework heel belangrijk. Een framework kan helpen om impliciet kennis van ervaren personen over te brengen en geeft starters een groeipad dat ze kunnen volgen om succesvol te worden binnen hun positie.

In het volgende hoofdstuk wordt daarom de methodologie beschreven waarmee de competenties worden verzameld, geprioriteerd en gevalideerd, en hoe daaruit een proof-of-concept leerplatform kan worden afgeleid.

3

Methodologie

Het uitwerken van een competentieframework en een leersite gebeurt in verschillende stappen. In dit hoofdstuk wordt kort toegelicht welke fasen en methoden in deze bachelorproef worden gevolgd om tot een onderbouwd en bruikbaar resultaat te komen.

3.1. Opbouw van de methodologie

Deze bachelorproef is opgebouwd uit vier gestructureerde fasen:

- **Fase 1:** Analyse voor het bepalen van de competenties en de requirements van de proof of concept.
- **Fase 2:** Opstellen en verfijnen van het competentieframework.
- **Fase 3:** Ontwikkelen van de proof of concept.
- **Fase 4:** Valideren van het competentieframework en de leersite.

3.2. Fase 1: Analyse van competenties en requirements

In de eerste fase wordt onderzocht welke kennis en vaardigheden relevant zijn voor startende mainframeontwikkelaars, met een specifieke focus op PL/1 en CICS. Daarnaast wordt in deze fase bepaald aan welke functionele en inhoudelijke vereisten de proof of concept moet voldoen. Deze analyse vormt de basis voor de verdere uitwerking van zowel het framework als het leerplatform.

In deze fase wordt gebruikgemaakt van de voorgaande literatuurstudie en bestaande competentieframeworks. Dit zorgt voor een actueel beeld van de nodige competenties binnen de bedrijfswereld en vormt het theoretische fundament van het onderzoek.



Figuur 3.1: Competentie framework.

3.3. Fase 2: Opstellen van het competentieframework

3.3.1. Opstellen initieel competentieframework

Op basis van de literatuurstudie en de analyse van bestaande frameworks wordt eerst een initieel competentieframework opgesteld. Dit framework bevat een eerste gestructureerd overzicht van de competenties die nodig zijn voor een startende PL/I en CICS developer binnen een professionele z/OS rol. De competenties worden gegroepeerd per domein, zoals programmeerkennis, transactionele verwerking, werken met mainframeomgevingen en algemene professionele vaardigheden.

Dit eerste framework dient als vertrekpunt voor verdere validatie in het werkveld. Het heeft als doel om de bevindingen uit de literatuur om te zetten in een bruikbaar en overzichtelijk model dat later verfijnd kan worden aan de hand van feedback van professionals. Door al in deze fase een duidelijke structuur op te stellen, wordt het eenvoudiger om gaten te identificeren in het framework en om te bepalen welke competenties essentieel zijn voor een eerste instapniveau.

Daarnaast maakt dit initiële framework het mogelijk om al een eerste koppeling te maken met het toekomstige leerplatform. Elke competentie kan vertaald worden naar mogelijke leerdoelen, leerinhouden of oefeningen. Hierdoor vormt het framework niet alleen een theoretisch overzicht maar ook een concrete basis voor de verdere ontwikkeling van de proof of concept.

3.3.2. Interviews met professionals uit het werkveld

Ter validatie en verfijning van het initiële framework worden interviews afgenomen bij verschillende professionals van Euroclear die actief zijn binnen PL/I en CICS

teams. Deze interviews worden voorbereid op basis van de voorlopige lijst van competenties die uit de literatuurstudie is afgeleid. Door deze voorbereiding kunnen de interviews niet alleen gebruikt worden om de voorgestelde competenties te valideren, maar ook om ruimte te laten voor verdere discussie en bijkomende inzichten vanuit de praktijk.

Er is bewust gekozen voor een semigestructureerde aanpak. Daardoor kunnen gerichte vragen gesteld worden over het voorlopige framework, terwijl de geïnterviewde professionals ook de mogelijkheid krijgen om extra competenties, aandachtspunten en praktijkervaringen aan te brengen die niet expliciet uit de literatuur naar voren kwamen.

De centrale vragen tijdens deze interviews waren de volgende:

- Zijn de voorgestelde competenties correct, of zijn er aanpassingen nodig?
- Welke belangrijke competenties ontbreken nog in het framework?
- Wat zijn volgens u de grootste tekortkomingen bij nieuwe starters?
- Welke soft skills zijn essentieel voor een startende professional in deze functie?

Op basis van de resultaten van deze interviews kan het voorlopige competentieframework verder worden verfijnd en bijgestuurd. Zo wordt het framework niet alleen gebaseerd op literatuur, maar ook afgestemd op de actuele verwachtingen en noden van het werkveld.

3.4. Fase 3: Ontwikkelen van de proof-of-concept

Na het vaststellen van de competenties volgt de daadwerkelijke ontwikkeling van de proof of concept (PoC). De methode in deze fase bestaat uit het selecteren van de juiste technologieën en het structureren van de leerinhoud.

In plaats van direct te programmeren worden er eerst selectiecriteria opgesteld voor de technologiestack, waarbij flexibiliteit, uitbreidbaarheid en de kosten centraal staan. Op basis van deze criteria wordt een selectie gemaakt van geschikte webtechnologieën om een interactieve leersite te bouwen.

Daarnaast is de methodologische keuze gemaakt om theorie, oefeningen en quizzen strikt van elkaar te scheiden en de content datagedreven in te laden zodat het platform eenvoudig schaalbaar blijft. De concrete uitwerking van deze proof of concept, de gekozen architectuur en de finale inhoudelijke modules worden uitgebreid geanalyseerd en gepresenteerd in Hoofdstuk 5.

3.5. Fase 4: Valideren van het competentieframework en de PoC

Tenslotte wordt het competentieframework gevalideerd. Deze validatie gebeurt aan de hand van zowel het framework zelf als de bijbehorende leersite. Hiervoor

zullen gesprekken worden gevoerd met professionals uit het werkveld en met inhoudelijke experts, om na te gaan of er nog gaten zitten in de inhoud of in de opbouw.

Tijdens deze validatiefase wordt niet alleen gekeken naar de volledigheid van het framework, maar ook naar de bruikbaarheid van de leersite als ondersteunend leer-materiaal. Daarbij wordt nagegaan of de leerinhoud voldoende aansluit bij de verwachte voorkennis, of de oefeningen relevant en haalbaar zijn en of de opbouw van inhoud logisch en doeltreffend is.

Op basis van de feedback van deze professionals kunnen gerichte aanpassingen worden doorgevoerd aan zowel het competentieframework als de proof of concept. Vervolgens kunnen studenten van HOGENT in het huidige en volgende academiejaar met dit framework en de leersite oefenen. Op die manier kan worden geëvalueerd of deze aanpak hen effectief de gewenste basiscompetenties bijbrengt en hen beter voorbereidt op de leerstof en praktijk die volgt.

Daarnaast kan deze testfase ook waardevolle inzichten opleveren over de gebruiksvriendelijkheid van de website en de moeilijkheidsgraad van de oefeningen. Er kan tevens gepeild worden in welke mate studenten zelfstandig met het platform aan de slag kunnen. De resultaten van deze evaluatie vormen de basis voor verdere optimalisatie van het framework en de leersite.

4

Interviews

Een belangrijk deel van deze bachelorproef bestaat uit het afnemen van interviews met professionals uit het werkveld. Hierdoor kan een compleet beeld gevormd worden van de competenties die nodig zijn voor een beginnende PL/1 en CICS developer op z/OS.

In dit hoofdstuk wordt toegelicht hoe de interviews zijn opgebouwd en welke vragen aan de professionals zijn gesteld. Het doel van deze interviews is dan ook niet enkel het verzamelen van verschillende meningen, maar het identificeren van terugkerende inzichten en het verkrijgen van een helder beeld van de exacte competenties die nodig zijn in het werkveld.

4.1. Structuur en opbouw van de interviews

Voor de interviews werd er gekozen voor een semigestructureerde opbouw. Aan de hand van deze methode is er een grotere flexibiliteit, wat zorgt voor de voordelen van vaste vragen en voldoende ruimte geeft om eigen accenten te leggen. De flexibiliteit is ook belangrijk want het bepalen van competenties kan vaak niet gedaan worden door enkel het stellen van gesloten vragen maar heeft een bepaalde diepgang nodig die je in het gesprek kan bekomen.

De interviews zijn dan ook opgebouwd rond enkele kernvragen die rechtstreeks aansluiten bij de doelstellingen van de bachelorproef. Deze doelstelling is het in kaart brengen van competenties die een startende developer nodig heeft om succesvol te kunnen starten in een mainframeomgeving. Het stellen van deze vragen aan verschillende professionals in het werkveld kan ervoor zorgen dat er een terugkerend thema wordt gevonden. Aan de hand hiervan kan men dan een actueel competentieframework opbouwen.

Hiervoor zijn volgende vragen gesteld aan de professionals binnen Euroclear:

- **Vraag 1:** What slows down onboarding the most?

Aan de hand van deze vraag kan achterhaald worden welke aspecten en factoren het integreren met het team en opstarten van nieuwe werknemers vertragen. Ze geeft inzicht in de eerste problemen waarmee starters geconfronteerd worden en toont aan waar de grootste frustraties liggen in de eerste weken in het werkveld. Hieruit kan dan bepaald worden welke competenties al aanwezig moeten zijn die het onboardingproces kunnen ondersteunen bij nieuwe developers.

- **Vraag 2:** Which hard and soft skills do you wish juniors had before joining a mainframe team developing PL/I and CICS?

Met deze vraag werd er gepeild naar de technische en niet technische competenties die professionals graag aanwezig zien in junior developers. Uit de antwoorden kunnen we constateren welke voorkennis het meest waardevol is volgens professionals voor het starten binnen een PL/I en CICS development-team. Deze skills kunnen dan ook dienen als een concrete basis voor het maken van het competentieframework en de proof of conceptsite.

- **Vraag 3:** What system-level knowledge does a starting PL/I and CICS developer need?

Aan de hand van deze vraag kan er een beeld geschetst worden van de nodige historische kennis. Daarnaast kan er ook getoetst worden naar welke andere tools en technologieën belangrijk zijn om een basis in te hebben om effectief te zijn in de rol van developer.

Met deze hoofdvragen als basis werden meerdere gesprekken gevoerd met professionals van Euroclear. Elke professional legde eigen accenten afhankelijk van hun positie/specialisatie, maar er was wel een duidelijk patroon in de antwoorden. Vooral het belang van globale systeemkennis en de historische achtergrond van de tools kwam steeds terug. Daarnaast is het belang van een grondige kennis van de basisprincipes van programmeren niet te onderschatten.

4.2. Antwoorden van professionals

Uit de interviews kan geconstateerd worden dat het onboardingproces vaak minder wordt vertraagd door het ontbreken van theoretische kennis, maar door de complexiteit van de bestaande legacysystemen. Beginnende developers kunnen vrij snel de basis programmeerconcepten oppikken, maar deze kennis toepassen in de bestaande systemen is veel moeilijker. Hiervoor is kennis nodig van de historische groei van het systeem samen met de bedrijfsspecifieke manier van werken. In het eerste interview met een CICS specialist Caio kwam dit aspect prominent naar voren. Volgens hem is het essentieel dat een startende developer begrijpt waarom bepaalde technologieën vandaag nog bestaan en hoe deze zijn geëvolu-

eerd. Met deze kennis kunnen ze technologiekeuzes beter vatten en gaan ze sneller verbanden zien tussen oude en nieuwe systemen.

Volgens Caio kan men anders het complete beeld missen en denken dat iets verouderd is, ook al heeft het een heel belangrijke taak binnen het bedrijfsproces. Maar deze kennis wordt ook grotendeels opgedaan in het bedrijf zelf, want overal gebruiken ze andere conventies en systemen.

Hierdoor haalde Caio een tweede punt aan dat volgens hem van belang is te beheersen als een starter. Een algemene kennis van programmeerprincipes samen met kennis rond syntax. Caio duidde erop dat het niet vereist is om expertkennis te hebben rond PL/I en CICS, maar dat men wel moet beschikken over een degelijke basis in programmeren. Hier sprak hij van het begrijpen van variabelen, controlestructuren, foutafhandelingen en het lezen en begrijpen van bestaande code.

Die basis stelt een nieuwe starter in staat zich sneller aan te passen aan een specifieke omgeving. Men kan dan ook sneller focussen op het leren van een diepere technische kennis, een die je enkel kan opbouwen door ermee te werken in een professionele omgeving.

Het tweede interview met een PL/I expert Hervé bevestigde grotendeels de punten die Caio aankaartte. Ook hij duidde aan dat uitgebreide systeemkennis niet vooraf wordt verwacht aanwezig te zijn. Dit omdat het volgens Hervé het aanleren van dit meer iets is dat door ervaring in het bedrijf komt dan door zelfstandige studie. Wat hij wel belangrijk of zelfs als noodzakelijk zag was een sterke basis in programmeerprincipes, en het vertrouwd zijn met de syntax van PL/I. Volgens Hervé moet een starter wel in staat zijn om code te kunnen analyseren en bepaalde logische structuren te herkennen.

Naast het analyseren is het debuggen van code volgens Hervé misschien wel even belangrijk. Wie in staat is om foutmeldingen correct te analyseren en stap per stap jobuitvoer kan begrijpen, die starter zal in staat zijn om zelfstandig te leren uit eigen fouten. Debuggen is ook iets dat even belangrijk blijft, een ervaren coder gebruikt deze skill nog steeds dagelijks.

4.3. Conclusie van de interviews

Volgens de professionals is het dus niet realistisch om van een junior te verwachten dat die van dag één alle systeemkennis van het bedrijf kent. Veel van die kennis is gekoppeld aan hun specifieke bedrijfsomgeving en is iets dat aangeleerd wordt naarmate men meer reële toepassingen uitwerkt.

Maar een concrete basis in programmeerprincipes en kennis van de tools is een must. Het bezitten van deze basis stelt de starter in staat om zelfstandig bedrijfs-specifieke systeemkennis te leren.

5

Framework en PoC

Het grootste deel van mijn bachelorproef bestaat uit het opstellen van een competentie framework en bijhorende Proof of concept.

In het volgende deel wordt beschreven hoe het framework is opgesteld en met welke criteria deze beslissingen zijn genomen. Vervolgens wordt de structuur van de proof of concept samen met de verschillende hoofdstukken overlopen en wordt uitgelegd welke technieken gebruikt zijn voor het realiseren van deze leersite. Het doel van dit hoofdstuk is dus tweeledig, enerzijds wordt aangetoond hoe het framework tot stand is gekomen. Anderzijds wordt uitgelegd hoe het framework omgezet werd naar een praktisch leerplatform dat gebruikt kan worden om het basisniveau 1 van PL/1 en CICS development op IBM Z aan te leren.

5.1. Structuur en opbouw van het framework

Qua structuur leunt het competentieframework op gevestigde IBM frameworks, waaronder het IBM z/OS System Administrator Level 1 framework IBM, [2023a](#). Deze structuur garandeert een overzichtelijk document dat intuïtief in gebruik is voor zowel studenten als begeleiders in het werkveld. Het doel is een duidelijk profiel te schetsen van de vereiste junior kennis. Het framework is dan niet simpelweg een lijst met onderwerpen, maar een concrete opsomming van eindtermen na het afronden van de leerstof.

Om dit te realiseren is gekozen voor een opbouw met vier vaste pijlers:

- **Het domein:** het specifieke domein waartoe de competentie behoort.
- **De competentie:** het specifieke onderwerp of de vaardigheid die aangeleerd moet worden.
- **De leerdoelen:** de concrete en meetbare acties die aantonen wat de gebruiker begrijpt of kan toepassen.

- **De bronnen:** de literatuur en documentatie die gebruikt zijn om de competentie inhoudelijk te onderbouwen.

Aan de hand van deze structuur kan een overzichtelijk document opgesteld worden dat als basis kan dienen voor een proof of concept. Elke competentie wordt gekoppeld aan concrete leerdoelen zodat duidelijk is welke kennis verwacht wordt. Hierdoor is het framework niet enkel theoretisch maar ook praktisch toepasbaar.

5.2. Criteria voor het bepalen van de competenties

De competenties binnen het framework zijn bepaald aan de hand van meerdere criteria. Op deze manier sluit het framework zo goed mogelijk aan bij de kennis die een beginnende PL/I en CICS developer nodig heeft. Het Competentieframework PL/I / CICS developer op z/OS level 1 vertrekt vanuit volgende punten.

5.2.1. Basiskennis mainframe

Een eerste criterium is de basiskennis die nodig is om mainframeontwikkeling te begrijpen. Iemand die nog niet vertrouwd is met Z systems moet eerst enkele belangrijke begrippen begrijpen zoals z/OS, PL/I, CICS, datasets, jobs en batch/online transactieverwerking. Zonder deze basis is het moeilijk om de aangeleerde PL/I en CICS kennis juist te kunnen plaatsen binnen de ontwikkelingsomgeving.

5.2.2. Basis programmeerkennis

Een tweede criterium is het bezitten van een basis in programmeerkennis. Het gebruik van lussen, variabelen en datatypes zijn concepten die aan de basis liggen van elke programmeertaal. Hierdoor is het heel waardevol om deze te beheersen voor het programmeren met PL/I en CICS.

5.2.3. Praktijk gericht

Een derde criterium is een sterke focus op de praktijk. Het framework is bewust niet puur theoretisch opgebouwd. Er is juist veel aandacht besteed voor de daadwerkelijke vaardigheden die op de werkvloer nodig zijn, zoals het efficiënt kunnen lezen en veilig aanpassen van bestaande legacy code.

5.2.4. De link met CICS

Een vierde criterium is de integratie met CICS. Voor online transactieverwerking op mainframes wordt PL/I zelden geïsoleerd gebruikt, maar vrijwel altijd in combinatie met CICS. Daarom moeten concepten zoals transacties, tasks en datadoorgave via een COMMAREA of channels direct meegenomen worden in de basisopleiding.

5.2.5. Hedendaagse IT landschap

Een laatste criterium is de connectie met het hedendaagse IT-landschap. Legacy-systemen zoals IBM mainframes opereren binnen een groter geheel. Om te begrijpen waarom deze talen vandaag de dag nog steeds cruciaal zijn voor grote ondernemingen, moet er in het framework ook aandacht zijn voor modernisering en de manier waarop PL/I en CICS toepassingen integreren met moderne architecturen.

5.3. Nodige Competenties

Aan de hand van de interviews en literatuurstudie is bepaald dat het competentieframework bestaat uit dertien kerncompetenties. Deze zijn gegroepeerd in vijf categorieën die samen de basis vormen voor een Level 1 developer:

- **Fundamenten en Systeemcontext (Competentie 1-2):** Omvat algemene programmeerlogica en IBM Z basiskennis. Deze zijn noodzakelijk om de technische omgeving en de werking van het mainframe ecosysteem te begrijpen voordat specifieke talen worden aangeleerd.
- **PL/I Taalbeheersing (Competentie 3-4):** Behandelt de theoretische syntax en de praktische toepassing van PL/I. Dit is essentieel omdat PL/I de primaire programmeertaal is waarin de bedrijfslogica geschreven wordt.
- **CICS Transactieverwerking (Competentie 5-7):** Richt zich op CICS concepten en de integratie hiervan in PL/I programma's. Dit is onmisbaar voor het ontwikkelen en begrijpen van onlinetransactiesystemen op de mainframe.
- **Buildprocessen en Foutafhandeling (Competentie 8-10):** Behandelt de buildflow, JCL basis en systematische troubleshooting. Deze kennis is nodig om code succesvol te compileren en bugs te verhelpen.
- **Bedrijfscontext en Onderhoud (Competentie 11-13):** Focust op codeanalyse, de historische context en modernisering. Dit is cruciaal om de blijvende waarde van legacysystemen te begrijpen en ze veilig te kunnen onderhouden binnen moderne IT architecturen.

Het gemaakte framework vormt hiermee de inhoudelijke blauwdruk voor de leerstof, waarbij de dertien competenties de basis leggen voor het praktische gedeelte van deze proef.

5.4. Volgende niveau Competenties

Bewust is gekozen voor een scherpe afbakening op een basisniveau of junior profiel, dit waarborgt een haalbare leercurve. De overgang naar een volgend niveau aangeduid als Level 2 of medior profiel markeert de stap van conceptuele kennis naar praktische complexe applicatieontwikkeling. Waar niveau 1 zich richt op de

oriëntatie binnen het ecosysteem en het uitvoeren van kleine aanpassingen, vereist niveau 2 dat een developer zelfstandig volledige programma architecturen kan opzetten en optimaliseren. De volgende competenties vormen de kern van deze verdieping:

5.4.1. Geavanceerde PL1

Op Niveau 1 ligt de focus op de basis van PL/I zoals variabelen en simpele procedures. Niveau 2 gaat dieper in op het geheugenbeheer. Hier leert de ontwikkelaar werken met dynamic storage allocation via pointers en based of controlled variabelen. Daarnaast is het op dit niveau essentieel om complexe foutafhandeling te implementeren via ON-units, waardoor applicaties niet simpelweg crashen, maar gecontroleerd kunnen herstellen van runtime fouten. Dit zorgt voor de robuustheid die in een enterprise omgeving vereist is.

5.4.2. CICS verdieping

De stap naar Niveau 2 binnen CICS betekent de overgang van het lezen van code naar het bouwen van volledige online applicaties. Dit omvat de ontwikkeling van gebruikersinterfaces via BMS-maps (Basic Mapping Support) voor 3270-terminals. Verder leert de ontwikkelaar hoe hij via File Control rechtstreeks gegevens kan wegschrijven, updaten of verwijderen in VSAM bestanden. Ook het efficiënt gebruik van Temporary Storage Queues (TSQ) voor tijdelijke dataopslag gedurende een sessie is een cruciaal onderdeel van deze gevorderde kennis.

5.4.3. Geavanceerde tooling en diagnostiek

Waar een junior vaak afhankelijk is van eenvoudige "display" statements of compiler listings voor debugging, maakt een Niveau 2 developer gebruik van gespecialiseerde tools. Dit omvat het beheersen van de IBM z/OS Debugger om stap voor stap door complexe programmaflows te lopen. Daarnaast leert men op dit niveau hoe men systeem dumps moet interpreteren (bijvoorbeeld via IPCS) om de exacte oorzaak van een abend (applicatie crash) in de systeemkern te achterhalen.

5.4.4. Performantie optimalisatie en efficiëntie

In een omgeving waar miljoenen transacties per seconde worden verwerkt, is efficiëntie alles. Een Niveau 2 developer begrijpt de impact van code op het CPU verbruik. Men leert technieken om I/O operaties te minimaliseren en SQL queries of CICS commando's te tunen. De focus verschuift hier van "de code werkt" naar "de code werkt met een minimale footprint".

5.4.5. Security architectuur

Beveiliging op Niveau 2 gaat verder dan het simpelweg hebben van een inlogcode. De ontwikkelaar leert hoe applicatiecode moet integreren met externe security ma-

nagers zoals RACF. Dit houdt in dat men begrijpt hoe men autorisatie checks uitvoert binnen de code, zodat gevoelige bedrijfsdata enkel toegankelijk is voor geautoriseerde gebruikers of specifieke transactie ID's.

5.4.6. Modernisering

Niveau 2 slaat de definitieve brug naar moderne IT. De ontwikkelaar leert hoe hij legacy PLI en CICS programma's kan verbinden met RESTful API's via z/OS Connect. Hierdoor kan de ontwikkelaar copybooks (mainframe datastructuren) omzetten naar JSON formaat. Als resultaat kan legacy backend logica eenvoudig communiceren met moderne web applicaties en mobiele interfaces in een hybride cloud omgeving.

5.5. Van framework naar Proof of Concept

Het uitgewerkte framework wordt gebruikt als de functionele blauwdruk voor de proof of concept. Waar het framework antwoord geeft op de 'wat' vraag (wat moet men kennen?), beantwoordt de PoC de 'hoe' vraag (hoe leer je deze concepten?). De PoC is gerealiseerd als een laagdrempelige leersite die de student begeleidt in het verwerven van de gedefinieerde competenties.

Door de chronologie van het framework te respecteren en eerst universele logica te behandelen, dan de taal, gevolgd door debugging en context, wordt het brengen van te moeilijke leerstof vermeden. Dit verlaagt de drempel aanzienlijk voor IT'ers die nog nooit met een mainframe in contact zijn gekomen.

5.6. Structuur en opbouw van de PoC leersite

De leersite is modulair opgebouwd, waarbij elke pagina direct te herleiden is naar één of meerdere leerdoelen uit het opgestelde competentie framework. De vaste flow ziet er als volgt uit:

- **Fase 1:** Bepalen nodige technologieën voor ontwikkeling van basis site.
- **Fase 2:** Scope van content bepalen die in de leersite gaan worden opgenomen.
- **Fase 3:** Aan hand van literatuurstudie en interviews relevante inhoud toevoegen.

5.7. Bepalen van technologieën

In de opstartfase van dit project werd bewust tijd vrijgemaakt om de technologie keuzes af te toetsen aan het doel van de proof of concept. Het doel van de PoC was een toegankelijke leersite rond PL/1 en CICS met theorie, oefeningen en quizzen die vlot kan uitgebreid worden zonder extra hostingkost. De keuze ging daarom

niet naar de meest exotische of meest enterprise stack, maar naar een set technieken die snel resultaat opleveren en gemakkelijk onderhoudbaar zijn en tegelijk voldoende structuur bieden voor groei.

5.7.1. Selectiecriteria

De technologieën werden beoordeeld op volgende criteria:

- **Flexibiliteit en uitbreidbaarheid:** nieuwe cursussen en onderdelen (theorie, oefening, quiz) moeten toegevoegd kunnen worden zonder ingrijpende code wijzigingen.
- **Vertrouwdheid en leercurve:** de ontwikkeltijd is beperkt dus de stack moest aansluiten bij bestaande webkennis.
- **Kost en deploybaarheid:** aangezien er geen budget is voor hosting, moet de leersite goedkoop publiceerbaar zijn.
- **Performantie en gebruikservaring:** snelle laadtijden en een responsieve interface zijn belangrijk om de leersite aangenaam te gebruiken.
- **Veilig omgaan met leercontent:** leerinhoud bevat opmaak en soms embedded media dat moet veilig weergegeven worden in de browser.

5.7.2. Gekozen basis van webtechnologieën

De proof of concept is een webapplicatie en gebruikt daarom de klassieke web basis: HTML voor structuur, CSS voor styling en JavaScript voor interactiviteit. Deze keuze sluit aan bij het doelpubliek (leren via een browser, zonder extra installatie) en maakt het mogelijk om de site als statisch pakket te publiceren.

5.7.3. React als UI laag

Voor de gebruikersinterface werd gekozen voor React. De leersite bevat meerdere schermen en componenten waaronder cursusoverzichten, hoofdstukken, theoriepagina's, oefeningen en quizzes waarbij de UI dynamisch mee evolueert met de selectie van de gebruiker. React maakt het eenvoudiger om zulke herbruikbare componenten op te bouwen en toestand zoals de geselecteerde cursus of het thema consistent te beheren. Concreet in dit project:

- Cursussen worden in de interface als kaarten gepresenteerd en leiden naar een detailpagina met hoofdstuknavigatie.
- Het thema donker en licht wordt opgeslagen in localStorage zodat de voorkeur bewaard blijft.
- Verschillende onderdelen waaronder: theorie, oefeningen en de quiz delen dezelfde navigatiestructuur en layout wat component hergebruik stimuleert.

5.7.4. Vite als build en ontwikkeltool

Om de ontwikkelervaring snel en efficiënt te houden werd Vite gekozen als build-tool rond React. Vite staat bekend om snelle lokale ontwikkeling (snelle herlaadtijden en Hot Module Replacement) en levert ook een geoptimaliseerde productie build. Voor een proof of concept is dit essentieel, de focus blijft op functionaliteit en content zonder onnodige tijd te verliezen aan trage builds of complexe configuratie. Daarnaast past Vite goed bij het deploy doel, de output kan als statische site worden gehost.

5.7.5. Tailwind CSS voor consistente en schaalbare styling

Voor de vormgeving werd Tailwind CSS gebruikt aangevuld met PostCSS en Autoprefixer. Tailwind ondersteunt een utility first aanpak waarmee de interface snel consistent kan worden opgebouwd, zonder dat een grote hoeveelheid aparte CSS bestanden en naming conventies nodig is. Dit is nuttig in een leerplatform waar veel herhaalpatronen bestaan (panelen, knoppen, kaarten, navigatie) én waar het ontwerp later nog kan evolueren. In dit project is expliciet gekozen om dark mode te implementeren via de class strategie, aangezien deze aanpak uitstekend samenwerkt met Tailwind en eenvoudig te onderhouden is.

5.7.6. Content als JSON eenvoudig uitbreiden met nieuwe cursussen

Een belangrijk ontwerpdoel was uitbreidbaarheid, nieuwe leerinhoud moet toegevoegd kunnen worden zonder telkens React code aan te passen. Daarom wordt course content per cursus opgeslagen in een index.json bestand met een vaste structuur bestaande uit hoofdstukken, theorie, oefeningen en quizvragen. Deze aanpak heeft meerdere voordelen:

- **schaalbaar:** nieuwe cursussen of hoofdstukken toevoegen is in grote mate een inhouds taak;
- **versiebeheerbaar:** JSON bestanden zijn ideaal om in Git te beheren en te reviewen;
- **scheiding van verantwoordelijkheden:** UI en content blijven duidelijk gescheiden.

5.7.7. Markdown en veilige HTML rendering

Voor theorie inhoud wordt waar mogelijk Markdown gebruikt, omdat het laagdrempelig is om te schrijven en leesbaar blijft in bronvorm. Bij het renderen in de browser wordt Markdown omgezet naar HTML. Om te vermijden dat content onveilig of ongewenst HTML/JavaScript zou injecteren, wordt de output gefiltert met DOMPurify. Embedded video's worden gecontroleerd toegelaten, zodat de leersite veilig blijft terwijl interactieve content toch mogelijk is.

5.7.8. Lokale media zonder extra backend

De leersite ondersteunt videomateriaal door bestanden in de folder public/videos/ te plaatsen en die via bestandsnaam te refereren in de JSON content. Dit is een pragmatische keuze voor een proof of concept want er is geen aparte backend of streamingplatform nodig, en toch kan de leerervaring verrijkt worden met videos en afbeeldingen.

5.7.9. Waarom geen backend of database?

Er is bewust geen server side component toegevoegd in deze fase. De kern van de proof of concept is het valideren van de leerflow, de contentstructuur en de gebruikerservaring. Door te kiezen voor een statische front end met JSON content blijft de deploy eenvoudig en kosteloos, en kunnen iteraties snel gebeuren. Mocht later behoefte ontstaan aan voortgangsopslag of gepersonaliseerde trajecten dan kan er in een volgende fase alsnog een backend worden toegevoegd.

5.7.10. Conclusie technologieën

De uiteindelijke technologie set (React, Vite, Tailwind CSS, JSON content, Mark-down met filtering en statische media) sluit nauw aan bij de projectdoelen: snelle ontwikkeling, lage kost, veilige contentweergave en een structuur die het toevoegen van nieuwe PL/I en CICS leermodules zo eenvoudig mogelijk maakt.

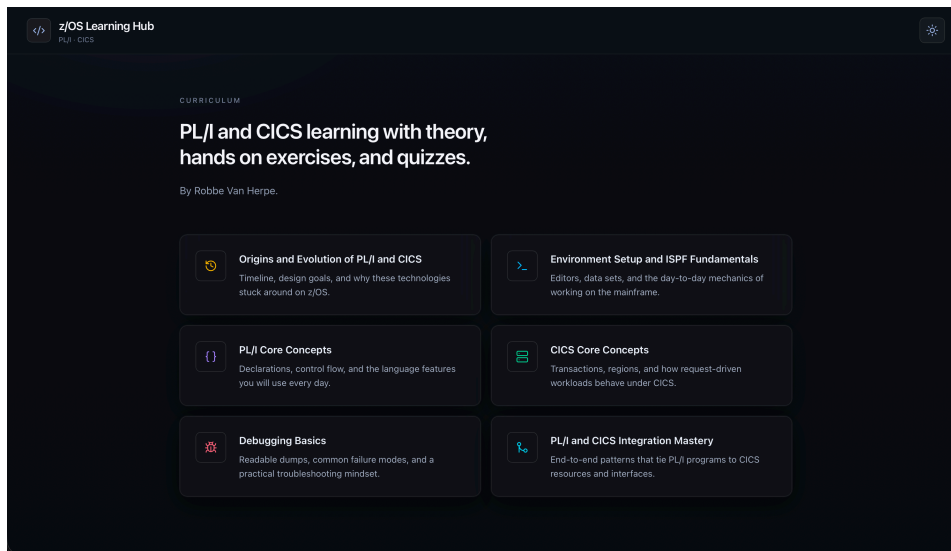
5.8. Verschillende modules in de leersite

De leersite is opgebouwd uit zes modules die samen een volledig en geleidelijk leertraject vormen rond PL/I en CICS. Elke module heeft een eigen plaats binnen het traject en werd opgenomen aan de hand van de feedback door professionals. Hierdoor leren studenten niet alleen afzonderlijke concepten zoals de PL/I syntax, maar begrijpen ze hoe die in een bredere mainframecontext met elkaar samenhangen en zijn ontstaan.

5.8.1. Origins and Evolution of PL/I and CICS

De module **Origins and Evolution of PL/I and CICS** werd opgenomen omdat het belangrijk is dat studenten eerst begrijpen waar deze technologieën vandaan komen en waarom ze vandaag nog steeds even relevant zijn.

Zonder de historische en conceptuele kennis zouden PL/I en CICS kunnen overkomen als irrelevante en eerder verouderde technologieën. Dit terwijl het net die systemen zijn die voor veel bedrijven nog steeds een zeer cruciale rol spelen binnen de financiële sector. Door te beginnen met de oorsprong en evolutie van deze technologieën krijgt de student een beter inzicht in hun plaats binnen enterprise computing en de redenen waarom organisaties er nog steeds op vertrouwen voor kritieke toepassingen.



Figuur 5.1: Webite Startscherm.

Deze module sluit direct aan op competentie 12.0 (Historische en enterprise-context) en competentie 13.0 (Modernisering en integratie) van het competentieframe­werk.

5.8.2. Environment Setup and ISPF Fundamentals

De module **Environment Setup and ISPF Fundamentals** legt de praktische basis om te leren werken met ISPF en Zowe Explorer. Het leren werken met deze code editors behandelt de competentie 2.0 (IBM Z basiskennis) en stelt de student in staat om zijn ontwikkelingsomgeving te beheersen. De student leert navigeren door schermstructuren en beheert datasets en jobs, wat essentieel is voor een goede start binnen het ecosysteem.

Legacy technologieën zoals ISPF hebben een steile leercurve voor nieuwe developers. Daarom wordt er in dit thema ook gefocust op alternatieven zoals Zowe Explorer. Studenten zijn moderne code editors zoals VSCode gewend en kunnen hierdoor efficiënter werken met deze tools. Daarom is deze module gefocust op beide manieren om de drempel zo laag mogelijk te maken voor nieuwe ontwikkelaars. Deze module zou als eindresultaat de student een goede basis moeten geven om te navigeren in hun ontwikkelingsomgeving, waardoor ze zich vervolgens kunnen focussen op het leren van PL/I en CICS.

5.8.3. PL/I Core Concepts

De module **PL/I Core Concepts** vormt de basis voor het programmeergedeelte van de leersite en is daarom cruciaal. Het behandelt enkele competenties, waaronder competentie 3.0 (PL/I taalbasis) en competentie 4.0 (PL/I programmeerpraktijk). Daarnaast wordt hier ook competentie 1.0 (Fundamenten in programmeren) opgefrist in een mainframecontext.

Studenten leren de specifieke syntaxis en declaraties van PL/I samen met de pro-

grammastructuren. Dit zorgt ervoor dat ze niet alleen code leren reproduceren maar ook begrijpen hoe PL/I logisch is opgebouwd binnen een professionele omgeving. Deze module vormt de bouwstenen waarop verder gebouwd kan worden om te werken binnen complexe legacysystemen als developer.

5.8.4. CICS Core Concepts

De module **CICS Core Concepts** is opgenomen omdat kennis van PL/I alleen niet volstaat wanneer men wil werken in transactionele mainframesystemen. CICS is een cruciaal onderdeel van veel bedrijfsapplicaties en bepaalt hoe transacties worden uitgevoerd. Daarnaast zorgt CICS voor het controleren van gegevensstromen en het verwerken van gebruikersinteracties met het systeem.

Het is dus belangrijk dat de student inzicht krijgt in de werking van CICS als systeemlaag naast het gebruik binnen PL/I zelf. De module legt uit wat transacties, tasks en resources zijn zodat de student de noodzakelijke conceptuele basis heeft om later PL/I programma's succesvol in een transactionele omgeving te laten draaien. Hiermee volbrengt deze module competentie 5.0 (CICS concepten).

5.8.5. Debugging Basics

De module **Debugging Basics** is belangrijk omdat leren programmeren of werken met mainframes niet alleen draait om het schrijven van correcte code maar ook om het kunnen herkennen, analyseren en oplossen van fouten. In deze module leren studenten onderscheid maken tussen de verschillende soorten fouten en maken ze kennis met het gebruik van IBM message manuals. Door deze eigenschappen kan de module gekoppeld worden aan competentie 10.0 (Foutanalyse en troubleshooting). In technische opleidingen is debugging een fundamentele vaardigheid, het geeft studenten een beter beeld tijdens het programmeren wanneer ze geconfronteerd worden met een probleem. Doordat hier een aparte module voor is voorzien zijn studenten beter gewapend tijdens hun eerste weken op de werkvloer wanneer ze zelfstandig problemen moeten oplossen. Deze module verhoogt dus niet alleen de technische kennis, maar ook het probleemoplossend denken van de student.

5.8.6. PL/I and CICS Integration Mastery

Tot slot is de module **PL/I and CICS Integration Mastery** opgenomen omdat je met al deze concepten apart niet het volledige totaalbeeld ziet. Hiervoor is een module nodig die al deze afzonderlijke domeinen samenbrengt tot een meer geavanceerde en realistische context. Nadat de student inzicht heeft verworven in zowel PL/I als CICS afzonderlijk, is het noodzakelijk om te tonen hoe beide in de praktijk met elkaar geïntegreerd worden.

Deze module integreert de competenties: 6.0 (CICS programmeerbasis), 7.0 (Gegevensuitwisseling in CICS), 8.0 (Build- en compileflow) en 11.0 (Codeanalyse en on-

derhoud) om tot één geheel te komen.

Studenten maken hier de overstap naar geavanceerde toepassingen, waarbij ze leren hoe programma's via de COMMAREA communiceren en hoe een volledige buildflow van translate tot link edit verloopt. Op deze manier leert de student niet alleen de onderdelen technisch te beheersen, maar ook de totale visie te behouden die nodig is voor professionele applicatieontwikkeling.

Deze module is daarom belangrijk als afsluiting van het leertraject, omdat studenten hierdoor leren om de geïsoleerde leerinhouden toe te passen in een meer realistische mainframeomgeving.

Samen zorgen deze zes modules voor een opbouw die logisch en opbouwend is.

De structuur van het leerplatform begeleidt de gebruiker stapsgewijs: van de initiële introductie en theoretische context, via de technische fundamenteën, naar de uiteindelijke praktische implementatie. Hierdoor ontstaat er een leeromgeving die studenten aanzet om verder te gaan met de theoretische kennis die wordt opgedaan. Als resultaat bekomen we studenten die de brug kunnen slaan tussen abstracte theorie en de complexe realiteit van een professionele mainframeomgeving, en zo met vertrouwen hun eerste stappen als junior developer kunnen zetten.

5.9. Bepalen opbouw van de modules

Nadat de verschillende modules zijn gekozen is het bepalen van de opbouw van deze onderdelen cruciaal. Er moet een logische opbouw zijn zodat studenten hun kennis steeds kunnen opbouwen en direct kunnen testen.

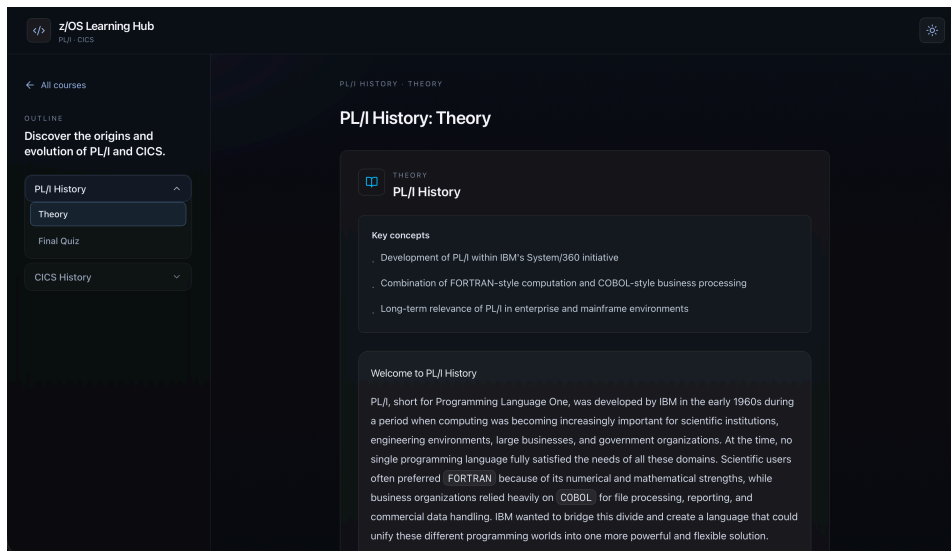
De opbouw van elk hoofdstuk binnen de leersite is bewust opgedeeld in drie vaste onderdelen: een theoretisch gedeelte, een oefengedeelte en een afsluitende quiz. Deze structuur werd gekozen om de gebruiker stapsgewijs door het leerproces te begeleiden. Eerst wordt de nodige kennis aangebracht, vervolgens krijgt de gebruiker de kans om die kennis toe te passen, en tot slot wordt nagegaan in welke mate de inhoud daadwerkelijk begrepen en beheerst wordt.

Op deze manier wordt de student ondersteund bij ieder onderwerp en ervaren ze steeds kleine mini succesjes die hen motiveren doorheen het hele proces.

5.9.1. Theorie in modules

In het eerste onderdeel **de theorie**, wordt de inhoud van het hoofdstuk uitgebreid toegelicht.

Hier worden de kernconcepten en technische principes behandeld die noodzakelijk zijn om het onderwerp goed te begrijpen. De leerstof is gestructureerd opgebouwd waardoor studenten hun kennis kunnen opbouwen en verdiepen in onderwerpen die voorgaand werden besproken. Hierdoor kan er eerst een brede basis worden aangebracht en later worden verdiept. Dit is belangrijk omdat de materie binnen PL/I, CICS en mainframeontwikkeling vaak complex is en een sterke onderlinge samenhang vertoont. Doordat de theorie op een logische manier en in fases



Figuur 5.2: Webite theorie module.

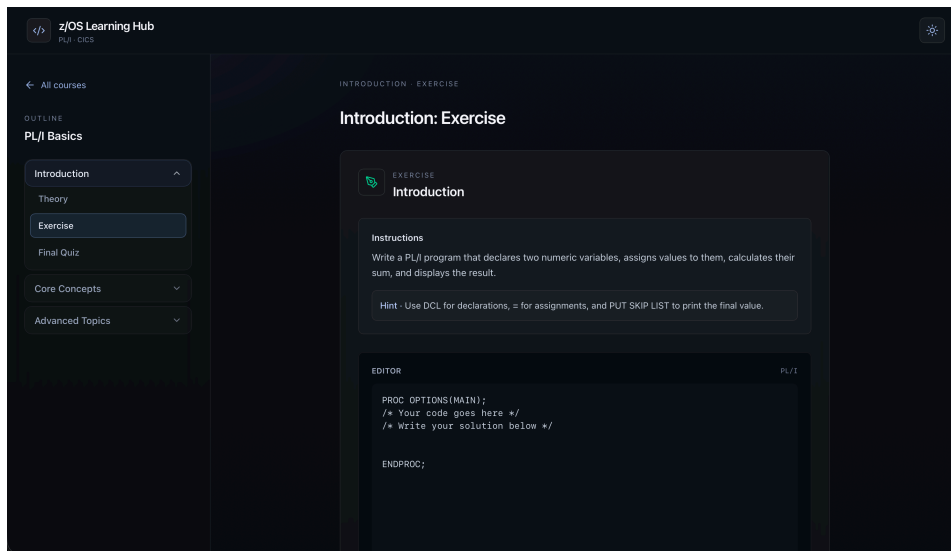
wordt aangebracht kan de student een correct beeld krijgen van alle technologieën en hoe ze samenwerken.

Daarnaast wordt de theoretische uitleg aangevuld met concrete voorbeelden en extra bronnen. Die voorbeelden helpen om abstracte concepten toegankelijker te maken en tonen hoe de theorie zich vertaalt naar praktische situaties. De toegevoegde bronnen bieden studenten de mogelijkheid om zich verder te verdiepen in specifieke onderwerpen. Ook kunnen ze aan de hand van deze bronnen de leerstof herbekijken wanneer iets niet helemaal duidelijk was. Dit voorkomt dat de leersite een gesloten leeromgeving is, maar geeft studenten juist de mogelijkheid tot zelfstandig leren. Studenten met meer interesse of voorkennis kunnen zich zo verder ontwikkelen terwijl anderen de basisinhoud op hun eigen tempo kunnen verwerken.

5.9.2. Oefeningen in modules

Na het theoretische gedeelte volgt het onderdeel met **de oefeningen**.

In deze fase wordt de student aangezet om de aangeleerde concepten toe te passen aan de hand van praktische voorbeelden. De oefeningen sluiten rechtstreeks aan bij de inhoud die eerder in het hoofdstuk werd behandeld en vormen daardoor de verbinding tussen theorie en praktijk. Dit onderdeel is essentieel, omdat technische kennis pas echt betekenis krijgt wanneer zij in de praktijk wordt gebruikt. Door zelf programmeeroefeningen te maken en problemen op te lossen wordt de student gedwongen om actief na te denken over de leerstof in plaats van deze enkel passief op te nemen. De oefeningen hebben daarbij niet uitsluitend een controlerende functie, maar maken deel uit van het leerproces. Ze geven de gebruiker de mogelijkheid om te experimenteren en fouten te maken. Hieruit kunnen ze inzicht krijgen in welke onderdelen reeds goed begrepen zijn en waar nog moeilijkheden



Figuur 5.3: Webite Oefeningen module.

liggen. Vooral bij het leren programmeren is deze actieve verwerking van groot belang.

Vaardigheden zoals logisch redeneren, syntaxis correct toepassen en fouten herkennen worden immers niet alleen ontwikkeld door theorie te bestuderen, maar vooral door herhaaldelijk te oefenen. De oefeningen zorgen er dus voor dat de leerstof niet oppervlakkig blijft maar effectief wordt omgezet in bruikbare vaardigheden.

5.9.3. Quiz in modules

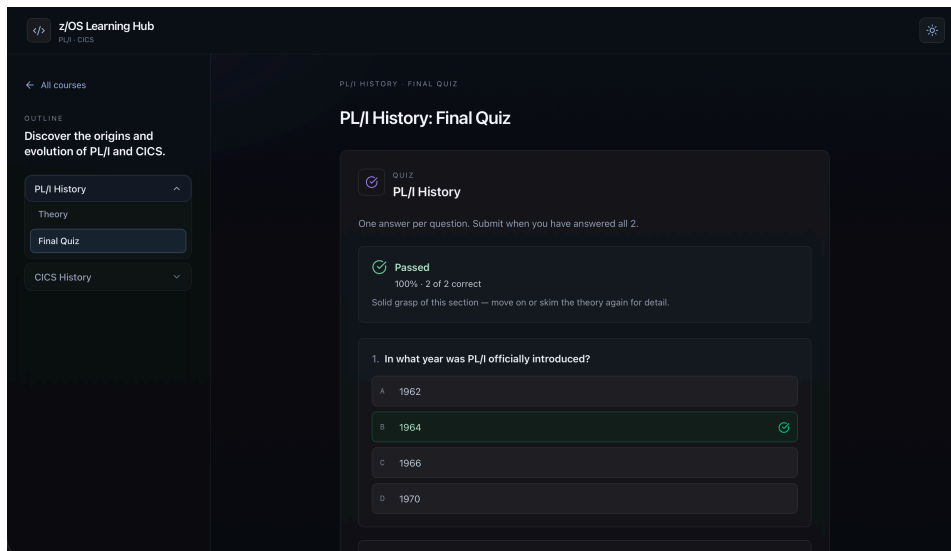
Als afsluiting van elk hoofdstuk is er **een finale quiz** voorzien.

Deze quiz heeft als doel om de volledige inhoud van het hoofdstuk te evalueren. Waar de oefeningen zich eerder richten op het inoefenen van afzonderlijke onderdelen, brengt de quiz alle behandelde leerstof samen in één evaluatiemoment.

Hierdoor krijgt de student de kans om na te gaan of niet alleen de afzonderlijke concepten zijn begrepen maar ook het totaalbeeld duidelijk is. Dit maakt de quiz tot een belangrijke tool om te kunnen meten in welke mate de leerstof van het hoofdstuk is verwerkt en opgenomen.

De finale quiz vervult bovendien een motiverende en structurerende rol binnen de leersite. Ze biedt de student een duidelijk eindpunt van het hoofdstuk en stimuleert hen om de leerstof grondig door te nemen in plaats van ze enkel oppervlakkig te bekijken. Ook maakt de quiz direct zichtbaar welke onderdelen al goed gekend zijn en welke nog verdere herhaling vereisen.

Voor een online leersite is zo'n evaluatiemoment bijzonder waardevol om de student te helpen zelfstandig het eigen leerproces op te volgen. De quiz vormt dus niet alleen een afsluitende test, maar ook een hulpmiddel om bewust en doelgericht verder te leren.



Figuur 5.4: Webite quiz module.

Door deze drie onderdelen in elk hoofdstuk te laten terugkeren, ontstaat een herkenbare en gestructureerde leeromgeving. De gebruiker weet telkens wat er verwacht wordt en doorloopt steeds dezelfde logische opbouw: eerst begrijpen, daarna toepassen, en tot slot evalueren. Die herhaling in de modulestructuur verhoogt de gebruiksvriendelijkheid van de leersite en ondersteunt het leerproces. Het zorgt ervoor dat de focus niet ligt op het zoeken naar informatie of het navigeren door de inhoud, maar op het effectief verwerven van kennis. Zo draagt de gekozen opbouw rechtstreeks bij aan de academische kwaliteit en de praktische bruikbaarheid van de volledige leeromgeving.

6

Conclusie

Dit laatste hoofdstuk van de bachelorproef bespreekt de bijdragen aan het onderzoeksdomein, reflecteert kritisch op de resultaten en formuleert een antwoord op de centrale onderzoeksvraag. Aansluitend worden openstaande vragen en aanbevelingen voor verder onderzoek geformuleerd.

6.1. Antwoord op de onderzoeksvraag

Vanuit de centrale onderzoeksvraag van deze bachelorproef kunnen enkele belangrijke conclusies worden getrokken. Deze onderzoeksvraag werd als volgt geformuleerd: *Hoe kan een competentieframework, in combinatie met een bijbehorend leerplatform, bijdragen aan een effectievere instroom en onboarding van startende PL/1 en CICS ontwikkelaars binnen een z/OS-omgeving?*

Op basis van de literatuurstudie en de interviews met professionals bij Euroclear, in samenhang met de ontwikkeling en validatie van zowel het framework als de proof of concept kan hierop een positief antwoord worden geformuleerd.

Een rolspecifiek en gevalideerd competentieframework biedt een structuur die de groei van opleiding naar het werkveld verkort. Het maakt het aanleren van complexe vakkennis toegankelijker voor starters en geeft hen een duidelijk groeipad. Voor organisaties biedt het een gestructureerde basis voor hun onboarding- en interne opleidingstrajecten.

Een framework in samenhang met een leerplatform zorgt voor een nog grotere meerwaarde aan het leertraject. Studenten kunnen direct hun theoretische kennis toepassen via gerichte oefeningen en kunnen zichzelf evalueren op hun vooruitgang. Dit zorgt voor een veel gestructureerdere manier van leren waarbij weinig tot geen externe interventie nodig is.

6.2. Bijdrage aan het onderzoeksdomein

De bijdrage van deze bachelorproef aan het onderzoeksdomein kan gesplitst worden in twee delen.

Op theoretisch vlak werd er een duidelijk overzicht opgebouwd van de relevante literatuur rond de skills gap in de mainframe-industrie en de onboarding van ontwikkelaars in legacy-omgevingen. Dit overzicht toonde aan dat er al bestaande frameworks zijn rond legacysystemen maar dat deze niet specifiek gericht zijn op de ontwikkelaarsrol voor PL/1 en CICS development op z/OS systemen. Deze bachelorproef vult deze leegte dan ook aan met een gevalideerd, rolspecifiek framework voor PL/1 en CICS-ontwikkelaars dat bestaat uit dertien kerncompetenties.

Op praktisch vlak werd er een proof of concept ontwikkeld die aantoont dat het competentieframework in de praktijk toegepast kan worden via de doelstellingen van een educatieve website. Het leerplatform kan direct gebruikt worden door studenten van Hogeschool Gent voor het aanleren van de belangrijkste competenties, maar ook als onboardingtool voor organisaties zoals Euroclear.

6.3. Kritische reflectie

Om een compleet beeld te krijgen van de resultaten is een kritische reflectie een heel belangrijk aspect. De initiële resultaten van de bachelorproef zijn positief, maar enkele beperkingen kunnen een verkeerd beeld geven.

6.3.1. Beperkingen van het onderzoek

Een eerste beperking die kan worden vastgesteld is de scope van de interviews bij professionals. De geïnterviewde professionals zijn allemaal van eenzelfde organisatie genaamd Euroclear. Hierdoor is het framework erg sterk gebaseerd op de verwachtingen van specifiek deze organisatie. Andere organisaties in verschillende sectoren kunnen andere verwachtingen stellen aan startende ontwikkelaars op hun systemen. Een mogelijke verbetering is het stellen van deze vragen bij verschillende organisaties, zodat er een meer globaal beeld gevormd kan worden.

Een tweede beperking is dat de effectiviteit van het leerplatform slechts beperkt getoetst kon worden. De leersite is gevalideerd door professionals en slechts door enkele studenten gedurende een gelimiteerde tijdsperiode. Hierdoor kunnen er nog geen zeer concrete conclusies getrokken worden uit deze testen. Dit zorgt ervoor dat de vraag openblijft of het leerplatform de studenten effectief en volledig zelfstandig door de leerstof begeleidt, of dat er alsnog externe ondersteuning nodig is. Deze vraag zou beantwoord kunnen worden door het uitgebreider valideren van de educatieve website bij studenten van HOGENT die de opleiding Mainframe Expert volgen.

6.3.2. Verwachte resultaten van het onderzoek

De verwachte resultaten van deze bachelorproef lopen in lijn met de uiteindelijke resultaten van het onderzoek. Er werd bevestigd dat de nood aan een rolspecifiek framework en een kennelijke skills gap zeker aanwezig zijn. Dit is niet erg verrassend, aangezien bestaande literatuur al op deze duidelijke signalen wees. Wat meer opviel tijdens de interviews was de nadruk die professionals legden op de historische en conceptuele kennis. Soft skills naast de hard skills werden als absolute succesfactoren aangekaart voor nieuwe starters. Dit aspect werd in bestaande frameworks nog te weinig besproken en is een duidelijk werkpunt voor toekomstige frameworks.

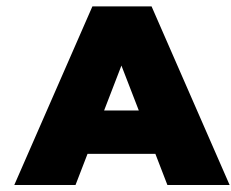
6.4. Aanbevelingen voor verder onderzoek

Deze bachelorproef zet ook aan tot opvolgend onderzoek door enkele nieuwe vragen die naar boven zijn gekomen.

Een zeer belangrijke vraag die gesteld kan worden, is welke langetermijnpact het framework en het leerplatform hebben op de effectiviteit van onboarding binnen legacy development teams. Een aanbeveling is om studenten van HOGENT gedurende een volledig academiejaar te laten experimenteren met het leerplatform en vervolgens hun onboardingproces binnen hun organisatie op te volgen. Met deze gegevens kan gericht nagegaan worden of het platform de overgang naar een mainframeomgeving daadwerkelijk heeft verkort en de productiviteit van starters heeft verhoogd.

Naast deze hoofdvraag kan er ook nog een tweede piste verkend worden, gebaseerd op de vraag of externe begeleiding nodig is. Het integreren van een AI-assistent binnen het leerplatform zou deze rol van externe begeleider kunnen invullen. Met de recente vooruitgang in taalmodellen rond mainframeapplicaties, zou een AI-assistent in staat kunnen zijn om intelligente feedback te geven op praktische oefeningen, en de student te begeleiden bij het debuggen van hun code zoals een ervaren begeleider dat zou doen. Dit zou de zelfstandigheid van de student enorm kunnen vergroten en tegelijkertijd de drempel om fouten te maken verkleinen.

Tot slot kan geconcludeerd worden dat het ontwikkelen van gestructureerde frameworks en opleidingstrajecten een pure noodzaak is voor het beperken van het vergrijzingsprobleem binnen z/OS-ontwikkeling. Organisaties die vandaag investeren in het implementeren van deze frameworks, bouwen een solide kennisbasis op bij nieuwe starters; de professionals die morgen de kritieke infrastructuur draaiende zullen houden.



Onderzoeksvoorstel

Het onderwerp van deze bachelorproef is gebaseerd op een onderzoeksvoorstel dat vooraf werd beoordeeld door de promotor. Dat voorstel is opgenomen in deze bijlage.

A.1. Inleiding

Binnen de IT-sector wordt voortdurend gekeken naar de nieuwste technologieën, onderwijsinstellingen passen hun curricula aan om deze nieuwe innovaties te kunnen ondersteunen. Door deze verandering in focus naar nieuwe technologieën is er voor mainframe-technologieën al een lange periode weinig aandacht. Hierdoor is er een groot tekort aan interesse gekomen voor deze industrie bij studenten in de IT, wat dan leidt tot een tekort aan nieuw talent om deze cruciale systemen operationeel te houden.

Maar eigenlijk is het beeld dat de mainframe verouderd is en niet meer relevant zeer onterecht. De huidige mainframe is een van de meest geavanceerde computers ter wereld en blijft een cruciaal deel van veel financiële instellingen en luchtvaartmaatschappijen, zij draaien hun meest kritieke activiteiten nog steeds op dit platform. Het grote probleem bevindt zich in het feit dat de gemiddelde leeftijd van professionals in dit vakgebied zeer hoog ligt, wat zorgt voor een zeker risico dat bedrijfsspecifieke kennis verloren gaat bij bedrijven zoals Euroclear. Wanneer een ervaren specialist met pensioen gaat zal er een groot deel van historische kennis en technische expertise verloren gaan, enkel wanneer ze deze kennis hebben kunnen doorgeven aan een nieuw talent kan dit vermeden worden.

A.1.1. Probleemstelling en onderzoeksvragen

We kunnen hieruit de centrale probleemstelling afleiden, er is **een groeiende kloof tussen de vraag naar nieuwe mainframeontwikkelaars en de hoeveelheid nieuw en**

opgeleid talent dat op de markt komt. Doordat onderwijsinstellingen bijna geen mainframe-specifiek aanbod hebben is de instroom van nieuw talent bijzonder laag.

De hoofdvraag van dit onderzoek is dan ook **Hoe kan een competentieframework voor PL/I en CICS op z/OS ontwikkeld worden dat als basis kan dienen voor een leerplatform voor startende mainframeontwikkelaars bij Euroclear?**

Deze vraag kan je niet beantwoorden met een simpel antwoord, daarom stel ik volgende deelvragen:

- Welke competenties zijn essentieel voor een beginnende PL/I en CICS developer binnen een z/OS-omgeving?
- Hoe kan een competentieframework worden opgebouwd dat aansluit bij de actuele noden van het werkveld?
- Hoe kan een proof-of-concept leerplatform op basis van dit competentieframework meetbaar bijdragen aan het verkleinen van de kenniskloof en het verkorten van de tijd tot inzetbaarheid van startende mainframeontwikkelaars?

A.1.2. Doelgroep

Deze bachelorproef is gericht op studenten van HOGENT en startende mainframe ontwikkelaars. Het kan hen een eerste basis geven voor een toekomstige job binnen dit vakgebied. Maar het biedt ook een zekere bijdrage aan bedrijven zoals Euroclear en HOGENT, zij kunnen dit framework toepassen om hun interne opleidingen te versterken en te valideren. Daarnaast kunnen organisaties de leerplatformsite gebruiken om hun nieuw talent op te leiden zodat zij beschikken over een zekere basis in PLI en CICS op z/OS.

A.1.3. Doelstelling

De doelstelling van mijn onderzoek is het ontwikkelen van een compleet en actueel competentieframework voor PLI/CICS developers op z/OS. Hiermee wil ik een proof-of-concept leerplatform ontwikkelen dat nieuw talent in staat stelt om zich te verdiepen in de materie, en zich voor te bereiden op een carrière in deze sector. Dit proof-of-concept leerplatform zou de tijd tot inzetbaarheid van startende mainframeontwikkelaars moeten verkleinen, wat zou zorgen voor een succesvolle conclusie van mijn onderzoek.

A.2. Literatuurstudie

De huidige stand van zaken is dat de mainframe-industrie een groot skillsprobleem tegemoetgaat en dit nu al ondervindt.

Het probleem van de vergrijzing binnen de mainframe industrie is al een tijdje zichtbaar en dit wordt weerspiegeld in enkele academische papers. In de paper

van Sharma (Sharma & Murphy, [2011](#)) en Ngo-Ye (Ngo-Ye & Choi, [2018](#)) wordt dan ook het tekort aan deze mainframe skills getoond. Dit tekort wordt deels veroorzaakt door de kloof tussen de academische instellingen en de bedrijfswereld. Maar een heel beperkt aantal universiteiten hebben een mainframe opleiding die studenten kunnen voorbereiden om een job in de mainframe industrie uit te gaan voeren, zelfs met de hoge vraag van financiële bedrijven voor deze profielen.

Daarnaast wordt in het onderzoek van Phillips (Phillips e.a., [2013](#)) ook gekeken naar een andere oorzaak van deze vergrijzing. Hier werd ook gezien dat de vergrijzing hand in hand gaat met een tekort aan motivatie voor studenten om mainframe-technologieën te leren. Studenten gaan sneller kiezen voor modernere technologieën en hierdoor is het aantal steeds erg beperkt.

Programma's zoals Master the Mainframe (momenteel IBM Z Xplore) van IBM zijn van levensbelang om nieuw talent aan te zetten om zich te verdiepen in deze technologie door de verschillende certificaten die je er kan verkrijgen. Door het gebruik van certificaten gaan nieuwe studenten sneller de stap zetten om het zelf eens te proberen.

Door dit alles is er een grote nood aan meer van deze leermogelijkheden, maar deze moeten dan wel van een sterke basis worden opgebouwd. Voor deze basis is een competentieframework zeer belangrijk en daar heeft IBM als grote belanghebbende van de mainframe dan ook enkele voorbeelden uitgebracht, waaronder IBM Z Systems Administrator Level 1 en Level 2 (IBM, [2023a](#), [2023b](#), [2023c](#)). Deze competentieframeworks zijn vooral gericht op de systeemkant van de mainframe en bevatten daardoor beperkte meerwaarde bij het ontwikkelen van leermateriaal voor startende Z/OS developers. Maar deze frameworks vormen wel een duidelijke richting en opmaak om het ontwikkelen van een competentieframework voor PLI/CICS developers te begeleiden.

Daarnaast heeft IBM nog een framework dat meer toepasselijk is het Application Developer on IBM Z competentieframework (IBM Apprenticeship Program, [2023](#)). Dit framework bevat alle competenties die een developer nodig heeft maar is een te algemeen framework. Het geeft in grote lijnen de belangrijkste onderdelen die een developer nodig heeft. Maar dit framework is sterk gebaseerd op COBOL en binnen bedrijven zoals Euroclear is een PLI/CICS framework veel interessanter. Een PLI/CICS developer op z/OS framework is dus nog steeds een gebied waar zo een overzicht mist.

A.3. Methodologie

Voor het succesvol uitvoeren van mijn onderzoek zijn er enkele deliverables die moeten worden voldaan binnen een bepaalde tijdspanne, deze omvatten:

- 3 feb 2026 - Indienen draft BP met inleiding literatuurstudie en methodologie.
- 4 mei 2026 - Indienen finale draft bachelorproef.

- 29 mei 2026 - Indienen bachelorproef.

Daarom ga ik te werk in volgende fases om deze deliverables op het juiste moment te kunnen leveren.

Overzicht van fases in Gantt Chart:



A.3.1. Literatuurstudie en interviews

In de eerste fase ga ik een literatuurstudie uitvoeren aan de hand van papers en artikels over PL1 en CICS zoals "CICS Transaction Processing on zOS: Core Concepts and Workflow" (Yalamanchili, 2021) en "Analyzing PL/1 Legacy Ecosystems: An Experience Report" (Sneed e.a., 2021). Daarnaast ga ik een analyse maken van voorgaande competentieframeworks en studies om tot een concrete lijst van requirements te bekomen. Deze voorlopige lijst wordt vervolgens verfijnd aan de hand van semi-gestructureerde interviews met twee tot drie professionals uit Euroclear en eventueel één of twee externe mainframe-experts. Ik kies hier voor semi-gestructureerde interviews, deze laten toe dat er gerichte vragen worden gesteld, maar geven ook ruimte voor bijkomende inzichten over nodige competenties. De verzamelde informatie wordt vervolgens geordend volgens het MoSCoW-principe om te komen tot een gestructureerde lijst met de belangrijkste requirements (Must haves en Should haves). Tot slot wordt deze lijst met requirements terug voorgelegd aan deze experts om gerichte feedback te krijgen en de requirements te valideren. Wanneer uit deze validatie blijkt dat er nog grote gaten zitten in mijn opgestelde lijst van requirements, dan zal deze moeten worden herzien. Voor deze fase verwacht ik dan ook een duur van ongeveer vier weken met als deadline 2 maart 2026.

A.3.2. Opstellen framework en PoC

De volgende fase van mijn onderzoek zal het opstellen van een compleet competentieframework bevatten dat als basis kan liggen bij het uitwerken van nieuw leer-materiaal. Dit is zeer relevant voor organisaties en beginnende ontwikkelaars want dit geeft hen een duidelijke leidraad om PL1 en CICS stap voor stap aan te leren, in plaats van te moeten werken met losse kennis en ongestructureerde documentatie. Het opstellen van dit framework ga ik doen aan de hand van de lijst met requirements, die alle noodzakelijke elementen bevat. Ik ga ook gebruik maken van de voorgaande competentieframeworks zoals het developer framework van IBM (IBM Apprenticeship Program, 2023) om een correcte opbouw en stijl te hebben in mijn competentieframework voor PL1/CICS developers.

Dit competentieframework zou dan als een sterke basis moeten dienen voor het uitvoeren van mijn volgende fases en organisaties in staat stellen om hun eigen leermateriaal te ontwikkelen specifiek voor PL1 en CICS. Concreet moet het framework het mogelijk maken om leerdoelen af te leiden en die om te zetten naar lessen of oefeningen.

Naast het uitwerken van dit framework ga ik ook aan de slag om een proof-of-concept leerplatform te maken dat gebaseerd is op dit competentieframework. Mijn doel met dit leerplatform is om na te gaan of er duidelijke leerdoelen en lessen kunnen worden afgeleid van het framework om hiermee de site van lesmateriaal te voorzien. Daarnaast wil ik studenten of beginnende ontwikkelaars de kans geven om zich te ontwikkelen in deze technologieën. Het leerplatform zal slechts een beperkt aantal features bevatten en vooral dienen als testomgeving voor het competentieframework.

Indien uit dit praktische voorbeeld blijkt dat het afleiden van leerdoelen en het opbouwen van lessen succesvol is, en als het leerplatform volgens experts voldoet aan de vooropgestelde eisen, dan beschouw ik mijn framework als een succes.

Voor het uitwerken van deze fase en het te bekomen van een competentieframework en proof-of-concept ga ik hoofdzakelijk gebruik maken van volgende technologieën.

- Excel voor het opstellen van het competentieframework.
- Github voor de versiecontrole van mijn poc site.
- HTML,CSS,JS voor de structuur, interactiviteit en opmaak van mijn poc site.
- Github Pages voor het hosten van mijn site.

Deze tweede fase zou klaar moeten zijn na zes weken met als deadline 4 mei 2026.

A.3.3. Rapporteren paper

Tenslotte in de laatste fase verwerk ik al deze resultaten in een paper als finale versie van mijn bachelorproef. Deze fase loopt eigenlijk gedurende het hele bachelorproefproces maar wordt gefinaliseerd in de laatste twee weken na het indienen van de draft. De deadline voor deze laatste fase is 29 mei 2026.

A.4. Verwacht resultaat, conclusie

Het verwachte resultaat van mijn bachelorproef is het opstellen van een actueel en compleet competentieframework voor PL1 en CICS developer op z/Os level 1. Met dit framework gaat er een duidelijk overzicht zijn van welke competenties nodig zijn om een succesvolle z/OS developer te zijn binnen cruciale bedrijven. Dit geeft andere onderzoekers en bedrijven de kans om dit framework als basis te gebruiken om hun leeromgeving te vormen voor toekomstige developers.

Aansluitend verwacht ik ook een proof-of-concept site gebaseerd op dit opgesteld competentieframework voor PLI en CICS developers. Aan de hand van deze learning-site zouden toekomstige z/Os developers een basis kunnen aanleggen in hun theoretische skills met PLI en CICS maar ook praktische aan de hand van hands on labs/examens. Deze combinatie zorgt ervoor dat ze een waardevolle toevoeging zijn binnen bedrijven die gebruikmaken van een mainframe.

Bibliografie

- Bradbury, J., & Hess, B. (2021). Fast Quantum-Safe Cryptography on IBM Z [Freely available via NIST CSRC]. *Third NIST Post-Quantum Cryptography Standardization Conference*. <https://csrc.nist.gov/CSRC/media/Events/third-pqc-standardization-conference/documents/accepted-papers/hess-fast-quantum-safe-pqc2021.pdf>
- Britto, R. (2017). *Strategizing and Evaluating the Onboarding of Software Developers in Large-Scale Globally Distributed Legacy Projects* (**publication**Nr. 9) [Doctoral thesis]. Blekinge Tekniska Högskola [URN: urn:nbn:se:bth-15197, DiVA id: diva2:1143875]. <https://bth.diva-portal.org/smash/get/diva2:1143875/FULLTEXT03.pdf>
- Buecker, A., Chakrabarty, B., Dymoke-Bradshaw, L., Goldkorn, C., Huguenbruch, B., Nali, M. R., Ramalingam, V., Thalouth, B., & Thielmann, J. (2016). *Reduce Risk and Improve Security on IBM Mainframes: Volume 1 Architecture and Platform Security* [Freely available via IBM Redbooks]. IBM Redbooks. <https://www.redbooks.ibm.com/abstracts/sg247803.html>
- Dau, A. T. V., Dao, H. T., Nguyen, A. T., Tran, H. T., Nguyen, P. X., & Bui, N. D. Q. (2024, augustus). XMainframe: A Large Language Model for Mainframe Modernization [Open access via arXiv]. <https://doi.org/10.48550/arXiv.2408.04660>
- Di Nardo, V., Fino, R., Fiore, M., Mignogna, G., Mongiello, M., & Simeone, G. (2024). Usage of Gamification Techniques in Software Engineering Education and Training: A Systematic Review. *Computers*, 13. <https://doi.org/10.3390/computers13080196>
- Ebbers, M., Kettner, J., O'Brien, W., & Ogden, B. (2012). *Introduction to the New Mainframe: z/OS Basics*. IBM Redbooks. <https://www.redbooks.ibm.com/redbooks/pdfs/sg246366.pdf>
- Ebbers, M., O'Brien, W., & Ogden, B. (2005). *z/OS Basics* [Publicly available textbook]. IBM International Technical Support Organization. <https://www.mabu.org/zosbasic.pdf>
- Granatella, G. (z.d.). AI-Driven Microlearning for Proactive DevSecOps: Closing the Security Skill Gap. *CEUR Workshop Proceedings*.
- Holmes, R., Allen, M., & Craig, M. (2018). Dimensions of experientialism for software engineering education. *Proceedings of the 40th International Conference on Software Engineering: Software Engineering Education and Training*, 31–39. <https://doi.org/10.1145/3183377.3183380>

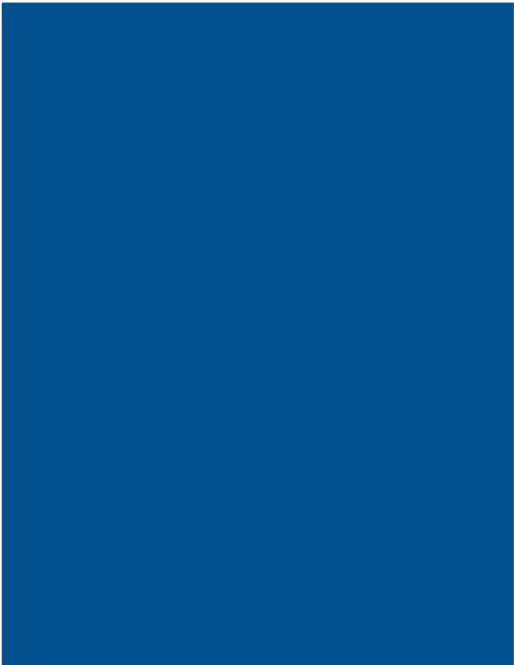
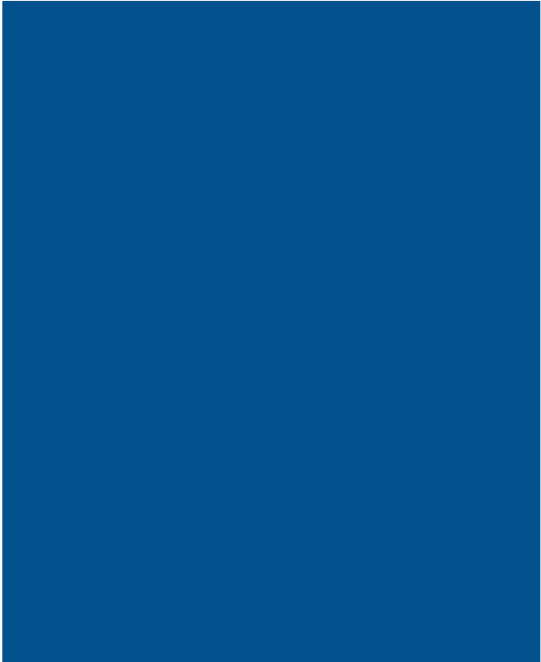
- Horswill, J., & Members of the CICS Development Team at IBM Hursley. (2000). *Designing and Programming CICS Applications*. O'Reilly Media. <https://books.google.be/books?id=73HYZLODkXEC>
- IBM. (2023a). *IBM Z Systems Administrator - Level 1 Competency Framework* (tech. rap.) (Focused on foundational z/OS system administration skills). IBM Training. <https://community.ibm.com/zsystems/uploads/document/slider/now8w2cj1q.pdf>
- IBM. (2023b). *IBM Z Systems Administrator - Level 2 Competency Framework* (tech. rap.) (Focused on advanced z/OS system administration and configuration). IBM Training. <https://community.ibm.com/zsystems/uploads/document/slider/c101df0smuh.pdf>
- IBM. (2023c, maart). *Mainframe System Administrator Apprentice Framework* [The occupational standards include the competency framework that outlines technical and professional competencies]. International Business Machines (IBM). Verkregen januari 5, 2026, van <https://www.ibm.com/downloads/cas/EBXQ9JDE>
- IBM Apprenticeship Program. (2023, maart). *Application Developer on IBM Z: Competency Framework* [O*NET CODE: 15-1132.00 (Software Developers, Application). Open source standard for apprenticeship and work-based learning programs.]. International Business Machines (IBM). Verkregen januari 5, 2026, van <https://community.ibm.com/zsystems/uploads/document/slider/ujdg9js84fj.pdf>
- Lascu, O., e.a. (2022). *IBM z16 Technical Introduction*. IBM Redbooks. <https://www.redbooks.ibm.com/redbooks/pdfs/sg248950.pdf>
- Lichtenau, C., Buyuktosunoglu, A., Bertran, R., Figuli, P., Jacobi, C., Papandreou, N., Pozidis, H., Saporito, A., Sica, A., & Tzortzatos, E. (2022). AI Accelerator on IBM Telum Processor. *Proceedings of the 49th Annual International Symposium on Computer Architecture*. <https://doi.org/10.1145/3470496.3533042>
- Ngo-Ye, T. L., & Choi, J. (2018). Teaching Students Mainframe Skills for the Niche Market: An Exploratory Proposal [Het onderzoekt het tekort aan mainframe-skills bij studenten en de noodzaak tot mainframe-onderwijs.]. *Proceedings of the Southern Association for Information Systems Conference*, 1–8. <https://aisel.aisnet.org/sais2018/>
- Phillips, B. K., Ryan, S., Harden, G., Guynes, C. S., & Windsor, J. (2013). Motivating Students to Acquire Mainframe Skills. *Proceedings of the 2013 Annual Conference on Computers and People Research (SIGMIS-CPR '13)*, 73–78. <https://doi.org/10.1145/2487294.2487308>
- Raju, J. (2024). The RAS Paradigm in Mainframe Systems: A Comprehensive Analysis of Design Principles and Operational Benefits [Open access, Creative Commons Attribution License (CC BY 4.0)]. *International Journal for Research in*

- Applied Science and Engineering Technology (IJRASET)*, 12. <https://doi.org/10.22214/ijraset.2024.64347>
- Raju, N., e.a. (2024). *IBM GDPS Family: An Introduction to Concepts and Capabilities*. IBM Redbooks. <https://www.redbooks.ibm.com/redbooks/pdfs/sg246374.pdf>
- Sharma, A., & Murphy, M. C. (2011). Teach or No Teach: Is Large System Education Resurging? [Analyseert het onderwijs in grote systemen (inclusief mainframes) en de aansluiting op marktbehoeften.]. *Information Systems Education Journal*, 9(4), 11–20. <https://files.eric.ed.gov/fulltext/EJ1145482.pdf>
- Sneed, H. M., Feigl, J., & Ferenc, R. (2021). Analyzing PL/1 Legacy Ecosystems: An Experience Report. *Proceedings of the 37th International Conference on Software Maintenance and Evolution (ICSME)*, 456–465. <https://doi.org/10.1109/ICSME52107.2021.00052>
- Sotaquirá-Gutiérrez, R., Beltran, L. M., & Garzon Ruiz, J. P. (2024). Hackathons as experiential learning platforms for engineering design skills. *Cogent Education*, 12. <https://doi.org/10.1080/2331186x.2024.2442187>
- Spruth, W. G. (2010). *System z and z/OS Unique Characteristics* (Technical Report Nr. WSI-2010-03). Wilhelm-Schickard-Institut für Informatik, Universität Tübingen. <http://hdl.handle.net/10900/49392>
- Yalamanchili, C. M. (2021). CICS Transaction Processing on zOS: Core Concepts and Workflow. 2, 1–13. <https://doi.org/10.5281/zenodo.15154786>



HO GENT

Robbe Van Herpe



**PL/1 & CICS Developer
on Z/OS Level 1**

Competentie framework



Competentie domein	Competentie	Leerdoelen	Bronnen
1.0 Fundamenten in programmeren	Beschikt over een sterke basis in programmeerlogica en algemene programmeerprincipes.	1. Kennis aantonen van de belangrijkste programmeerprincipes. 2. Basisconcepten begrijpen en toepassen, zoals: • variabelen • datatypes • operatoren • voorwaarden • lussen • functies/procedures 3. Eenvoudige programmeeroefeningen correct kunnen oplossen. 4. Foutmeldingen begrijpen, analyseren en zelfstandig oplossen.	https://www.w3schools.com/programming/index.php W3Schools Programming
2.0 IBM Z basiskennis	Begrijpt de basisprincipes van IBM Z, z/OS en mainframeontwikkeling	1. Uitleggen wat IBM Z en z/OS zijn en waarvoor ze gebruikt worden. 2. Basisbegrippen zoals datasets, jobs, JCL, loadmodules, batch en online processing herkennen. 3. De rol van mainframes binnen grote ondernemingen toelichten. 4. Het verschil begrijpen tussen ontwikkelen, compileren, linken en uitvoeren op IBM Z. 5. Basisnavigatie en oefeningen uitvoeren binnen een IBM Z-leeromgeving.	https://ibmzxplore.ibm.com IBM Z Xplore
3.0 PL/1taalbasis	Begrijpt de structuur, syntax en kernconcepten van PL/1programma's	1. De basisstructuur van een PL/1 programma herkennen en uitleggen. 2. PL/1 syntax correct lezen en interpreteren. 3. Werken met declaraties, datatypes, assignments, condities en iteraties. 4. Procedures en eenvoudige programmastructuren begrijpen.	IBM Enterprise PL/1 for z/OS Programming Guide https://www.ibm.com/docs/SSY2V3_5.3.0/com.ibm.ent.pl1.zos.doc/pg.pdf

		5. Het verschil tussen algemene programmeerlogica en PL/1 specifieke syntax herkennen.	
4.0 PL/1 program meerpraktijk	Kan eenvoudige PL/1 programma's schrijven, testen en verbeteren	1. Eenvoudige PL/1 programma's opbouwen volgens een duidelijke structuur. 2. Basislogica implementeren met condities, lussen en procedures. 3. Compileerfouten en syntaxisfouten herkennen en corrigeren. 4. Kleine aanpassingen uitvoeren in bestaande PL/1 code. 5. Output en gedrag van een PL/1 programma controleren en verklaren.	IBM Enterprise PL/1 for z/OS Programming Guide IBM Z Xplore
5.0 CICS concepten	Begrijpt de rol van CICS binnen online transactieverwerking op IBM Z	1. Uitleggen wat CICS is en waarom het gebruikt wordt. 2. De rol van CICS in online transactieverwerking beschrijven. 3. Basisbegrippen zoals transaction, task, program, region en resource correct gebruiken. 4. Het verschil tussen batchverwerking en online transactieverwerking uitleggen. 5. De relatie tussen transaction ID, CICS programma en task begrijpen.	IBM, What is CICS Transaction Server for z/OS? https://www.ibm.com/docs/en/cics-ts/5.5.0?topic=what-is-cics-transaction-server-zos IBM, Transactions in CICS https://www.ibm.com/docs/en/cics-ts/6.x?topic=applications-transactions-in-cics

6.0 CICS programmearbasis	Kan eenvoudige CICS programmalogica lezen en interpreteren bissen PL/1code	1. EXEC CICS instructies herkennen in bestaande code. 2. Basis-CICS commando's lezen en op hoofdlijnen verklaren. 3. Het onderscheid maken tussen normale PL/1logica en aanroepen naar CICS services. 4. Eenvoudige CICS programmaflows analyseren. 5. Begrijpen hoe een CICS programma samenwerkt met de runtimeomgeving.	IBM, CICS application programming languages and the CICS API https://www.ibm.com/docs/en/cics-ts/6.x?topic=applications-cics-application-programming-languages-cics-api IBM, Developing CICS Applications https://www.ibm.com/docs/en/SSJL4D_6.x/pdf/application-programming_pdf.pdf
7.0 Gegevensuitwisseling in CICS	Begrijpt hoe gegevens worden doorgegeven tussen CICS programma's en transactiestappen	1. Het doel van gegevensdoorgifte tussen programma's en transacties uitleggen. 2. COMMAREA herkennen en de basiswerking ervan verklaren. 3. Begrijpen hoe gegevens tussen opeenvolgende transactiestappen worden bewaard en doorgegeven. 4. Het verschil tussen COMMAREA en channels/containers op hoofdlijnen uitleggen. 5. Eenvoudige codefragmenten rond inter-program communication analyseren.	IBM, Sharing data across transactions https://www.ibm.com/docs/en/cics-ts/6.x?topic=applications-sharing-data-across-transactions IBM, Channels and containers IBM, Migrating from COMMAREAs to channels
8.0 Build- en compileflow	Begrijpt de buildflow van PL/1 en CICS programma's op IBM Z	1. Uitleggen hoe broncode wordt vertaald naar een uitvoerbaar programma. 2. De stappen translate, compile en link-edit op hoofdlijnen verklaren. 3. Begrijpen waarom CICS-code een translator-stap nodig heeft.	BM Enterprise PL/1 for z/OS Programming Guide https://www.ibm.com/docs/en/SSY2V3_6.2/pdf/pl_i_zos_6_2_programming_guide.pdf

		4. Het verband tussen source, object, loadmodule en runtime uitleggen. 5. Eenvoudige buildscenario's voor PL/1/CICS kunnen toelichten.	
9.0 JCL basis voor development	Kan eenvoudige JCL voor compile-, build- en uitvoeringsjobs lezen	1. De basisstructuur van een JCL-job herkennen. 2. JOB-, EXEC- en DD-statements op hoofdlijnen verklaren. 3. Datasets en jobstappen herkennen in compile- of buildjobs. 4. Compile-output koppelen aan de juiste jobstap. 5. Eenvoudige build- of uitvoeringsfouten situeren binnen de JCL-flow.	IBM z/OS Basic Skills IBM PL/1 Programming Guide
10.0 Foutanalyse en troubleshooting	Kan PL/1, CICS en buildfouten systematisch analyseren en oplossen	1. Het verschil uitleggen tussen compileerfouten, syntaxisfouten, runtimefouten, abends en CICS-responsmeldingen. 2. PL/1 compileroutput lezen en interpreteren. 3. Veelvoorkomende CICS-foutmeldingen en Abendcodes herkennen. 4. ESP en RESP2 gebruiken om CICS fouten te begrijpen. 5. Foutmeldingen koppelen aan mogelijke oorzaken in code, buildflow of runtimecontext. 6. Relevante IBM message manuals gericht gebruiken bij foutanalyse.	IBM Enterprise PL/1 for z/OS Messages and Codes https://www.ibm.com/docs/en/SSY2V3_6.1/pdf/mc.pdf IBM CICS Messages https://www.ibm.com/docs/en/cics-ts/6.x?topic=reference-cics-messages IBM RESP and RESP2 options https://www.ibm.com/docs/en/cics-ts/6.x?topic=conditions-resp-resp2-options

11.0 Codeanalyse en onderhoud	Kan bestaande PL/1 en CICS code functioneel lezen, analyseren en onderhouden	1. Bestaande PL/1 en CICS code op hoofdlijnen begrijpen. 2. Programmaflow, datastructuren, CICS aanroepen en foutafhandeling herkennen. 3. De relatie tussen transacties, programma's en gegevensuitwisseling analyseren. 4. Functioneel uitleggen wat een bestaand programma doet. 5. Kleine onderhouds aanpassingen voorbereiden met aandacht voor impact en risico.	IBM PL/1 documentatie IBM CICS Application Programming Guide IBM Z Xplore
12.0 Historische en enterprise-context	Begrijpt de geschiedenis, het gebruik en de blijvende relevantie van PL/1 en CICS	1. Uitleggen waarom PL/1 ontwikkeld werd en in welke context de taal ontstond. 2. Beschrijven waarvoor PL/1 traditioneel gebruikt wordt, zoals administratieve, wetenschappelijke en bedrijfskritische toepassingen. 3. Uitleggen waarom CICS ontwikkeld werd voor online transactieverwerking. 4. Voorbeelden geven van sectoren waarin PL/1 en CICS vandaag nog gebruikt worden, zoals banken, verzekeringen en overheid. 5. Het belang van onderhoud, continuïteit en modernisering van legacy-systemen toelichten.	IBM Enterprise PL/1 for z/OS Language Reference https://www.ibm.com/docs/en/SSY2V3_6.1/pdf/lrm.pdf IBM, What is CICS Transaction Server for z/OS? IBM CICS Transaction Server
13.0 Modernisering en integratie	Begrijpt op hoofdlijnen hoe PL/1 en CICS toepassingen passen binnen moderne IT-architecturen	1. Uitleggen waarom bestaande PL/1 en CICS-toepassingen vaak nog bedrijfskritisch zijn. 2. Begrijpen hoe legacy-systemen kunnen samenwerken met moderne applicaties. 3. Het verschil herkennen tussen onderhoud, migratie, integratie en modernisering. 4. Op hoofdlijnen toelichten hoe CICS kan functioneren binnen hybride enterprise-	IBM CICS Transaction Server https://www.ibm.com/products/cics-transaction-server IBM CICS-documentatie

		architecturen. 5. De impact van wijzigingen in legacy-omgevingen op bedrijfsprocessen begrijpen.	
--	--	---	--